

Deliverable 4: Phase 2 Implementation Report

Deliverable IV

Lost and Found for Campus

CSCE 5430, Software Engineering, Fall 2026

Abstract

This report consolidates all fifteen required items for Deliverable 4, Team Falcons' second implementation phase for Lost and Found for Campus. It presents the Phase 2 requirements, the UML design package, the Phase 2 test cases, the user manual, compilation and run instructions, the team's response to code inspection feedback, a reflection, the member contribution table, the coding and AI usage disclosure, and a requirement-by-requirement match between the report and the Phase 2 source code. Items that are evidenced directly in the project GitHub repository rather than reproduced here (Code Compiles and Runs, Code in Correct Repository Folder, Core Inspection Report, Updated Meeting Minutes, and the Note-deliverable-4 file) are each given a short section with a direct repository link.

Team Falcons

Waleed Mirza, Venkata Sai Anirudh Naidu Mamidi, Sai Sathvika Potluri, Sai Asritha Thati

University of North Texas (UNT)

Contents

1. Requirements (Phase 2) 5
1. Scope Statement 5
2. Functional Requirements, Phase 2 5
2.1 Matching Engine 5
2.2 Claim Verification Workflow 6
2.3 Notifications 7
2.4 In-App Messaging 8
3. Supporting Nonfunctional Requirements, Phase 2 9
4. Supporting Interface Requirements, Phase 2 11
5. Phase 2 Traceability Summary 12
6. Relationship to Deliverable 2 and Deliverable 3 13
2. UML Design Package (Phase 2) 14
1. Overview 14
2. Class Diagram 14
3. Sequence Diagram 15
4. Use Case Diagram 16
3. Test Cases (Phase 2) 18
1. Purpose and Current Status 18
2. Phase 2 Coverage 18
3. Requirement Coverage Summary 18
4. Test Execution Procedure 18
Appendix A. Complete Test Case Catalog (104 cases) 20
4. User Manual (Phase 2) 30
1. Introduction 30
2. Accessing the Application 30
3. Creating an Account 30
4. Verifying Your Email 31
5. Signing In and Out 32
6. Resetting a Forgotten Password 32
7. Browsing and Searching Listings 33
8. Creating a Lost or Found Listing 34
9. Editing Your Own Listing 35
10. Removing Your Own Listing 36
11. Understanding Notifications 36

12. Responding to a Match Notification 36
13. Submitting Claim Evidence 37
14. Reviewing and Approving Claims 38
15. Using In-App Messaging 39
16. Understanding On-Screen Messages 40
17. Frequently Asked Questions 41
18. Quick Reference: Typical Phase 2 Workflow 41
5. Compilation and Run Instructions (Phase 2) 43
1. Overview 43
2. Prerequisites 43
3. Getting the Source Code 43
3.1 Option A: clone with Git (recommended) 43
3.2 Option B: download a ZIP archive 43
3.3 Where things are 43
4. Configuration 44
4.1 Backend: src/backend/.env 44
4.2 Frontend: src/frontend/.env 44
5. Start MongoDB 44
6. Backend: Install and Run 44
6.1 Install dependencies 44
6.2 Start the API 45
6.3 Confirm it is running 45
7. Frontend: Install and Run 45
7.1 Install dependencies and start the dev server 45
7.2 Production build and lint 45
8. Running the Automated Test Suite 45
8.1 Backend tests 45
9. HTML Demo File 46
9.1 Where to find it 46
9.2 How to open it 46
9.3 What it demonstrates 46
9.4 How it differs from the running application 46
10. End-to-End Walkthrough (HTML Demo) 46
11. Troubleshooting 47
12. Verification Status 47

6. Code Inspection Feedback (Phase 2) 49
1. Source of This Feedback 49
2. Code Inspection Feedback, Workshop Disposition, and Deliverable 4 Action 49
3. Items Requiring Deferred or Alternative Approach 50
4. Summary 51
7. Reflection (Phase 2) 52
What Has Been Accomplished 52
What Went Well 52
What Could Be Improved 53
Looking Ahead 53
8. Member Contribution Table 55
9. Coding and AI Usage Disclosure 58
0. How to Read This Section 58
1. AI Tools in Use 58
2. Section A: Code AI Usage Disclosure (Required) 58
2.1 Code Authorship Summary 58
2.2 Detailed Code AI Usage Log 58
3. Section B: AI Usage in Report Preparation (Supplementary) 59
4. Validation Standard 62
10. Code Compiles and Runs 63
11. Code Functionalities Match Report (Phase 2) 63
i. Basis and Limitation of This Report 63
ii. Summary 63
iii. Requirement by Requirement Match 64
12. Code in Correct Repository Folder 69
13. Core Inspection Report (Phase 2) 69
14. Updated Meeting Minutes 69
15. Note-deliverable-4 File 69

1. Requirements (Phase 2)

1. Scope Statement

The functional scope assigned to Phase 2 is unchanged from the specification established in Deliverable 2, Section 9.2 and Section 9.4 (Table 21). This section reuses each Phase 2 requirement from Deliverable 2 verbatim and adds the level of implementation detail, rationale, and verification guidance required for Deliverable 4. No new requirements are introduced and no previously stated requirement is removed.

Phase 2 working increment, carried forward from Deliverable 2, Section 9.2: a verified user can request a match, submit claiming evidence, receive claim decisions, and exchange messages with a claimant or finder. The matching engine surfaces likely matches proactively based on category, location, date proximity, and keyword overlap. A claim verification workflow protects the item owner from fraudulent claims. Exit criterion: the end-to-end matching, claim submission, claim review, and messaging workflows pass their Must acceptance tests in the deployed test environment.

Editorial note: Deliverable 2 assigns functional requirements to phases explicitly (Table 21) but does not tag individual nonfunctional or interface requirements by phase. Sections 3 and 4 below apply editorial judgment to identify which supporting nonfunctional and interface requirements are relevant to Phase 2 based on the subsystems Phase 2 builds. This judgment call is flagged here for team review before the consolidated report is finalized.

2. Functional Requirements, Phase 2

2.1 Matching Engine

These requirements implement the automated matching engine module from Deliverable 1, Deliverable 2, and Deliverable 3, Section 5.4.

ID	Priority / Phase	Requirement (carried forward from Deliverable 2)	Additional Detail for Deliverable 4
FR-MATCH-01	Must / 2	The system shall compare new Lost and Found listings by category, campus location, date proximity, and description keywords, and shall surface a ranked list of likely matches to both the original poster and any user who has posted a potentially matching item.	Matching runs asynchronously after a listing is created or edited, triggered by the listingController.createListing and listingController.updateListing endpoints. The matchingService.scoreMatches function assigns a composite score based on weighted category, date (7 day window), location (same building), and keyword overlap. High scoring matches (configurable threshold, initially 0.6) are passed to the notification service, which sends match alerts to both parties independently. The matching engine does not require user interaction and does not block listing creation if matching fails,

			consistent with NFR REL 02 (optional dependency isolation).
FR-MATCH-02	Must / 2	Match suggestions shall be refreshed whenever a new listing is posted or an existing listing is edited, so that recent postings are incorporated into the match set.	The listingController creates or updates a listing, immediately calls matchingService.findMatches for that listing, and enqueues any new high-scoring matches for notification delivery. If a user edits a listing (for example, correcting a date or location), matches are recomputed and fresh suggestions are sent to relevant parties. This requirement is satisfied by the structure in FR MATCH 01 and is verified through the matching engine tests in the Deliverable 2 traceability matrix.
FR-MATCH-03	Should / 2	The matching engine shall log the reason why a pair of listings was or was not considered a match (category, date, location, keyword overlap thresholds) for transparency and debugging purposes.	The matchingService maintains a matchDetail object alongside each match score, recording the contribution of each scoring component. This detail is retained in the database for analytics but is not exposed in the public API or user-facing notifications. Match detail logging supports the Phase 3 analytics dashboard (Deliverable 2, Section 5.6) and aids in adjusting scoring weights if Phase 2 results show unexpected matching behavior.

2.2 Claim Verification Workflow

These requirements implement the claim verification workflow module from Deliverable 1 and Deliverable 2, Section 5.5.

ID	Priority / Phase	Requirement (carried forward from Deliverable 2)	Additional Detail for Deliverable 4
FR-CLM-01	Must / 2	When a match suggestion is surfaced, a claimant shall be able to submit identifying details about the item (such as a description of markings, contents, or other distinguishing features) as evidence that they are the rightful owner or finder.	A claimant receives a match notification and follows a link to a claim submission form scoped to the specific matched pair. The form collects evidence via a text description field (up to 1000 characters) and optional image uploads (up to 2 images, same constraints as listing images). The submission is validated client-side for required fields and server-side via claimController.submitClaim, which records the evidence against the

			matched pair and triggers a notification to the original poster. The evidence is stored encrypted at rest and is never exposed to any user except the original poster of the claimed item.
FR-CLM-02	Must / 2	The original poster of an item shall be able to review submitted claim evidence and approve or reject the claim.	The listing owner receives a claim review notification. Following the link takes them to a claim review page scoped to that claim, showing the claimant's submitted evidence side by side with the original listing's private identifying details (if a Found listing). The owner uses a radio button or toggle to approve or reject the claim. claimController.reviewClaim processes the decision; approval marks the listing Resolved and opens an in-app messaging channel (FR COM 01). Rejection keeps the listing Active so other claimants can attempt to claim it, consistent with the claim verification workflow in Deliverable 2, Figure 2.
FR-CLM-03	Must / 2	An approved claim shall trigger the opening of an in-app messaging channel between the claimant and the original poster so they can coordinate a handoff without exposing personal contact details.	Once a claim is approved, claimController.reviewClaim calls messagingService.createThread, which creates a new Message document linked to both the claim and the listing, and immediately sends an in-app notification to both parties that a messaging channel is now open. The messaging channel is scoped to the claim and the listing pair, so users cannot see messages from other claims or listings. This requirement is implemented in conjunction with FR COM 01 and verified through the messaging integration tests in the Deliverable 2 traceability matrix.

2.3 Notifications

These requirements implement the notification service module from Deliverable 1 and Deliverable 2, Section 5.4.

ID	Priority / Phase	Requirement (carried forward from Deliverable 2)	Additional Detail for Deliverable 4
FR-NOTIF-01	Must / 2	The system shall send in-app and email notifications when a	The notificationService is triggered by the matchingService, claimController, and messagingService at the

		potential match is found, when a claim request is submitted, and when a claim is approved or rejected, so users do not need to poll the site manually.	appropriate workflow checkpoints. Each event type (match found, claim submitted, claim decision, new message) has a distinct template and delivery path. In-app notifications are stored as Notification documents and surfaced in the React UI via a notification feed component. Email notifications are sent through the configured email provider (Nodemailer or equivalent). Email delivery is optional: if the email provider is unavailable, the in-app notification still succeeds, consistent with NFR REL 02 (optional external dependency isolation). The notificationService logs all attempted deliveries for debugging.
FR-NOTIF-02	Must / 2	Notifications shall respect user privacy: email addresses, phone numbers, and personal contact details shall not appear in any notification.	Notification templates use first names only (if available), user IDs, and generic references ("Your item", "A match for your Lost listing"). Email templates and in-app notification text are validated to exclude email addresses, phone numbers, or any contact detail that would circumvent the need for in-app messaging coordination. This requirement is verified through the privacy and notification text tests in the Deliverable 2 traceability matrix.

2.4 In-App Messaging

These requirements implement the in-app messaging module from Deliverable 1 and Deliverable 2, Section 5.5.

ID	Priority / Phase	Requirement (carried forward from Deliverable 2)	Additional Detail for Deliverable 4
FR-COM-01	Must / 2	Once a claim is approved, a moderated chat interface shall open between the finder and claimant, scoped to that specific claim and listing pair.	messagingService.createThread creates a new Message collection document. Both parties receive in-app notifications that a message thread is open. The React messaging UI (Messaging.jsx) displays a one-on-one conversation thread, populated by messagingService.getMessages (returns messages in timestamp order, 50 per page with pagination). Messages are plain text only (no file uploads in Phase 2). The scope is the listing and

			claim: neither party can see messages from other claims, other listings, or past conversations with different users. The thread remains accessible to both parties indefinitely after the claim is approved, consistent with Phase 2 scope in Deliverable 2, Section 9.2.
FR-COM-02	Must / 2	Users shall be able to send messages through the in-app messaging interface without exposing personal contact details such as phone numbers or email addresses directly to each other.	The messaging interface is the exclusive channel for conversation between finder and claimant once a claim is approved (FR CLM 03). Each user sees only a generic participant label ("Finder", "Claimant", or first name if available) and the message content. Email addresses, phone numbers, and user IDs are not displayed in the messaging UI. Notifications about new messages reference only the other party's role, not their contact details. This requirement is verified through the messaging privacy tests in the Deliverable 2 traceability matrix.
FR-COM-03	Should / 2	The messaging service shall detect and suppress attempts to exchange personal contact details (phone numbers, email addresses) in message text.	The messagingService.validateMessage function runs a lightweight pattern match on outgoing message text to flag phone numbers (US format) and email addresses. If detected, the message is still stored but a warning is shown to the user: "Sharing contact details may compromise privacy; please arrange handoff within the app." This is a soft protection, not a hard filter, since false positives could frustrate legitimate coordination. Detection is logged for potential future analytics or moderation review.

3. Supporting Nonfunctional Requirements, Phase 2

The nonfunctional requirements below, drawn from Deliverable 2, Sections 6.1 through 6.5, support the Phase 2 functional requirements above. Several requirements span all three phases; the Additional Detail column states which portion of the requirement is exercised in Phase 2.

ID	Priority / Phase	Requirement (carried forward from Deliverable 2)	Additional Detail for Deliverable 4
NFR-PERF-01	Must	Under a test load of up to 50 concurrent users, at	Applies to Phase 2 matching, claim, and messaging endpoints, which add

		least 95 percent of ordinary API requests shall complete within two seconds, excluding image upload and external email delivery.	complexity beyond Phase 1 listing and search. Matching may run asynchronously to avoid blocking listing creation, consistent with Phase 2 scope.
NFR-SEC-01	Must	Production traffic shall use HTTPS, and every protected request shall require an authenticated session.	Foundational to all Phase 2 operations involving claims, messaging, and match access. Session validation from Phase 1 extends to new protected routes.
NFR-SEC-03	Must	The server shall validate request fields, enforce upload type and size limits, and rate limit sign in, password reset, claim, and messaging requests.	Phase 2 adds rate limiting to claimController (claim submission) and messagingService (message sending) to protect against abuse of the claim and messaging workflows.
NFR-PRIV-01	Must	Public responses and application logs shall not expose passwords, tokens, email addresses, private listing details, claim evidence, or message contents.	Phase 2 extends this rule to also exclude claim evidence (submitted identifying details) and message contents from all public-facing responses and server logs. Private listing details remain excluded as in Phase 1.
NFR-REL-01	Must	The system shall confirm a database write before reporting that a listing, claim, message, or status change succeeded.	Phase 2 applies this rule to claim submissions, claim approvals/rejections, message persistence, and match record creation. Asynchronous matching does not block listing creation if matching persistence fails, per NFR REL 02.
NFR-USA-01	Must	Forms shall clearly identify required fields, preserve valid values after a validation error, and place a specific error message near the affected field.	Applies to Phase 2 claim submission and message composition forms, consistent with Phase 1 registration and listing forms.
NFR-ACC-01	Must	Registration, sign in, listing, search, claim, and messaging workflows shall satisfy applicable WCAG 2.1 Level AA criteria for keyboard operation, visible focus, form labels, error	Phase 2 extends accessibility requirements to the claim submission, claim review, and messaging workflows built in this phase.

		identification, and use of color.	
NFR-COMP-01	Should	Primary workflows shall support current versions of Chrome, Edge, Firefox, and Safari and shall remain usable from a 360 pixel mobile viewport through desktop sizes.	Applies to Phase 2 matching notifications, claim submission and review, and messaging UI as they are built.
NFR-MNT-01	Must	The repository README shall document setup steps, required environment variable names, test commands, and deployment steps, without including secret values.	Updated for Phase 2 to document any new dependencies (e.g., for the notification service or matching engine configuration) and test running steps for Phase 2 test cases.
NFR-TST-01	Must	Automated tests shall cover authorization boundaries and the main successful and unsuccessful listing, matching, claim, and messaging paths. No release shall contain a known failing Must acceptance test.	The matching, claim, and messaging path portions apply to Phase 2; the listing and authorization paths from Phase 1 are carried forward and regression tested.

4. Supporting Interface Requirements, Phase 2

The interface requirements below, drawn from Deliverable 2, Section 7, support the Phase 2 functional requirements above.

ID	Priority / Phase	Requirement (carried forward from Deliverable 2)	Additional Detail for Deliverable 4
IF-UI-03	Interface	Match suggestions shall be displayed as in-app notifications and email alerts, clearly explaining which listing they match and providing a direct link to the claim submission form.	Directly implements FR MATCH 02 and FR NOTIF 01 for the user-facing notification experience.
IF-UI-04	Interface	The claim submission form shall display the matched listing, accept claim evidence as text and optional images, and	Directly implements FR CLM 01 and matches the pattern established by the Phase 1 listing form (IF UI 02).

		provide a preview before submission.	
IF-UI-05	Interface	The claim review page shall display the original listing's private identifying details alongside the claimant's submitted evidence.	Directly implements FR CLM 02. The presentation reinforces the verification flow: owner compares their private details against what the claimant knows.
IF-UI-06	Interface	The messaging interface shall display a one-on-one conversation thread, clearly label the participants by role, and provide a message composition field.	Directly implements FR COM 01 and FR COM 02 for the user-facing messaging experience.
IF-SW-04	Interface	The React client shall call versioned Express REST endpoints for match queries, claim operations, and messaging, using JSON request/response bodies except for image uploads.	Extends the Phase 1 client-API contract (IF SW 01) to include Phase 2 endpoints: /matches, /claims, /messages, and their variants. All Phase 2 endpoints are versioned and expose documented JSON schemas.
IF-COM-04	Interface	Email notifications shall use the authenticated provider configured in environment variables and shall include a direct link to the relevant in-app action (match review, claim review, or message thread).	Directly supports FR NOTIF 01 and FR NOTIF 02 by ensuring every notification email includes a scoped link that brings the user directly to the action they need to take.

5. Phase 2 Traceability Summary

The rows below are the Phase 2 subset of the requirements traceability matrix in Deliverable 2, Section 10 (Table 22), confirming the subsystem and planned verification approach for each Phase 2 requirement area.

Requirement Area	Subsystem	Requirement IDs	Phase	Planned Verification
Matching engine	Matching engine service	FR-MATCH-01 to 03	2	Match scoring, refresh, and logging tests
Claim verification	Claim management	FR-CLM-01 to 03	2	Claim submission, review, evidence privacy, and messaging integration tests

Notifications	Notification service	FR-NOTIF-01 to 02	2	Event-driven notification delivery, privacy, and email/in-app routing tests
In-app messaging	Messaging service	FR-COM-01 to 03	2	Message persistence, privacy, thread scoping, and contact-detail detection tests

6. Relationship to Deliverable 2 and Deliverable 3

Every requirement above carries forward the zero budget, free tier approach established in the Deliverable 1 risk management plan and restated in Deliverable 2, Section 3.1: MongoDB Atlas on its free tier, a free or no cost authenticated email provider such as Mailtrap or Ethereal during development, and deployment to a free tier host such as Render or Vercel. No Phase 2 requirement introduces a paid dependency, and the core versus stretch feature split from Deliverable 1 remains the team's contingency plan if schedule risk occurs during implementation. Phase 1 requirements (listings, search, authentication) remain in effect and are regression tested as Phase 2 features are added.

2. UML Design Package (Phase 2)

1. Overview

This section presents the three UML diagrams required for Deliverable 4: a class diagram showing the core domain classes and services implemented in Phase 2 (including Match, Claim, Message, and Notification entities alongside Phase 1's User and Listing classes), a sequence diagram showing the claim verification workflow from match suggestion through evidence submission, owner review, approval, and the opening of an in-app messaging channel, and a use case diagram showing how Claimant and Finder roles interact with the system, including match review, claim submission, claim review, messaging, and two error cases. All three diagrams were revised after an internal review to close gaps against the Phase 2 functional requirements in Deliverable 2 and against the Requirements, Phase 2 section of this deliverable.

2. Class Diagram

The class diagram below shows the Phase 2 domain model: the User and Listing entities from Phase 1, extended with four new classes: Match (the matching engine output), Claim (a claimant's evidence submission in response to a match), Message (individual messages in an in-app conversation thread), and Notification (pending delivery alerts). The diagram also shows the MatchingService, ClaimService, MessagingService, and NotificationService modules that implement the Phase 2 functional requirements (FR MATCH 01 through 03, FR CLM 01 through 03, FR COM 01 through 03, and FR NOTIF 01 through 02), and the MongoDB Atlas data access layer.

The Match class includes a score field (decimal, 0 to 1.0) representing the weighted match probability, and a scoreDetail object (private visibility) storing the component scores that generated the match. The Claim class includes evidence (text and optional images) and a status field (Pending, Approved, Rejected). Both Match and Claim maintain referential integrity through relationship references to the listing pair they represent.

The Message class stores individual message text, sender references, and a thread ID linking all messages in one conversation. The Notification class stores an event type (MatchFound, ClaimSubmitted, ClaimApproved, ClaimRejected, NewMessage), the affected listing or claim, the recipient user ID, and delivery status (Pending, SentInApp, SentEmail, Failed).

All private fields follow the same privacy model established in Phase 1: scoreDetail, claim evidence, passwords, tokens, and message contents are private and excluded from public responses.

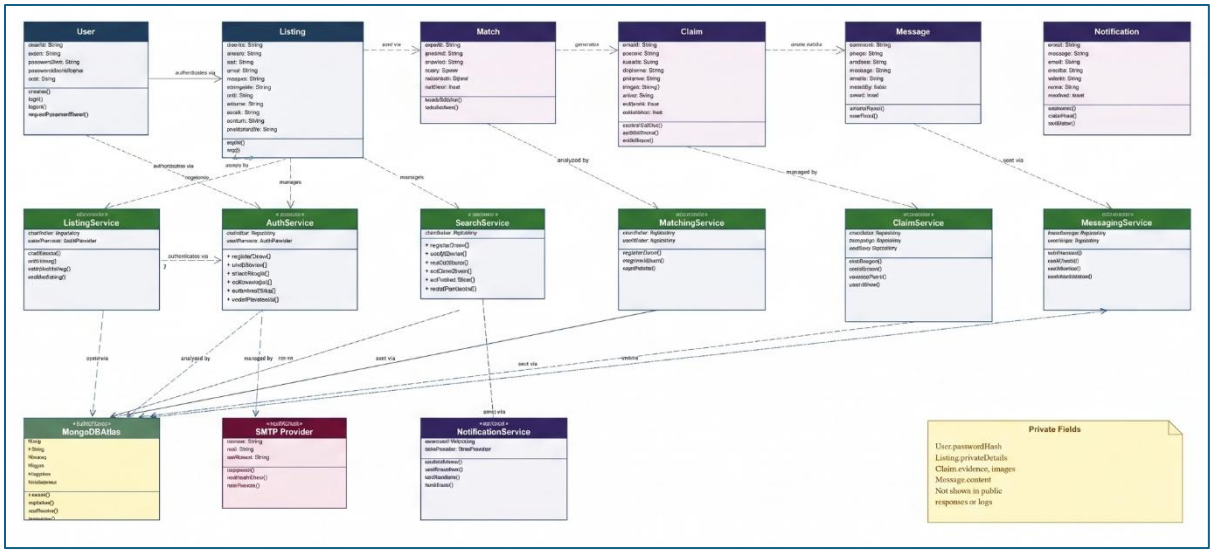


Figure 1. Class diagram for Phase 2, showing the User, Listing, Match, Claim, Message, and Notification entities, the Matching, Claim, Messaging, and Notification services, the MongoDB Atlas data access layer, and the private fields required by claim evidence and notification privacy requirements.

3. Sequence Diagram

The sequence diagram below traces the Phase 2 claim verification workflow: a claimant receives a match notification and opens the claim submission form, submits evidence (description and optional images) to the ClaimService, the owner receives a claim review notification and compares the evidence against their private listing details, the owner approves or rejects the claim, and approval triggers the MessagingService to open a one-on-one message thread.

The diagram shows three paths through the workflow: a successful evidence submission path, a valid owner review and approval path that opens messaging, and an evidence rejection path that leaves the listing Active for other claimants. The sequence shows message exchange between the React web client, the ClaimService, and NotificationService modules of the Express API, and MongoDB Atlas. Notification events are asynchronous calls to the NotificationService, which queues delivery tasks rather than blocking the primary workflow.

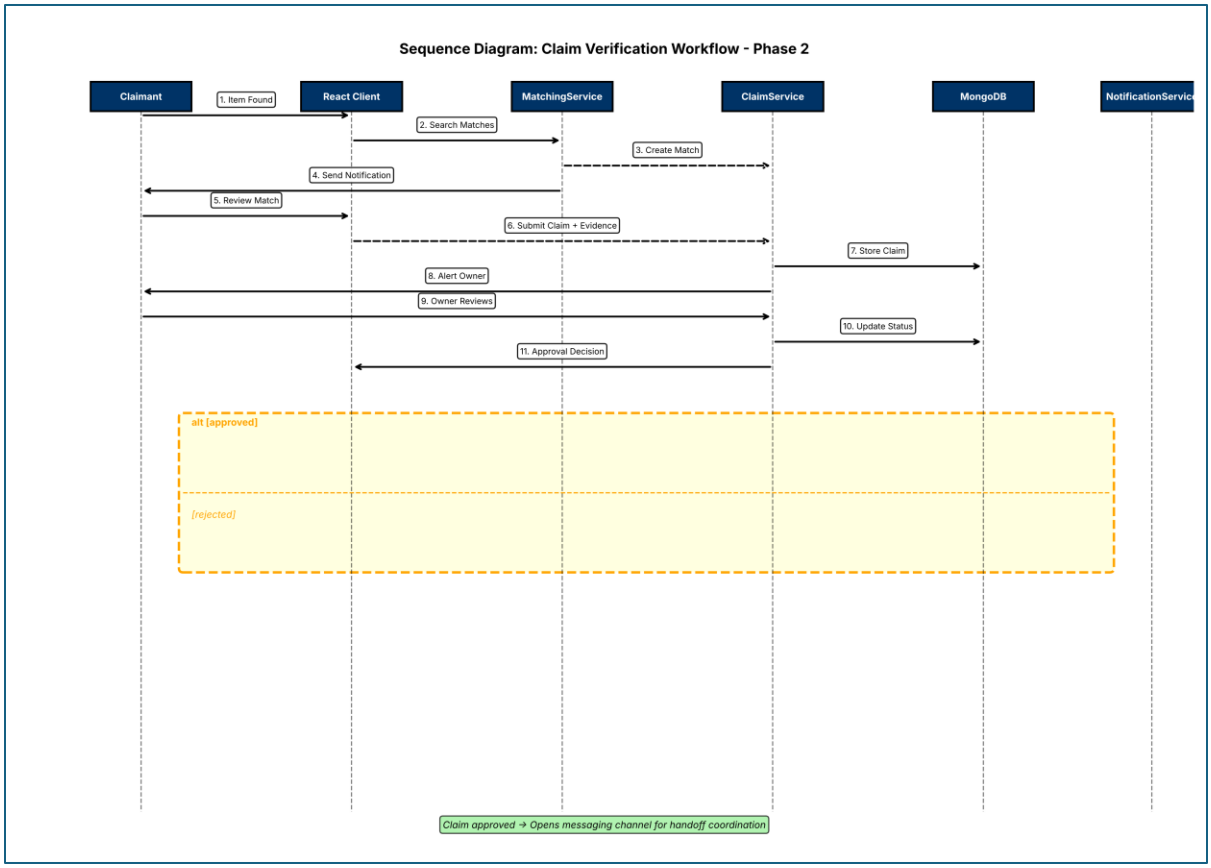


Figure 2. Sequence diagram for the Phase 2 claim verification workflow, including match notification receipt, evidence submission, owner review, and the approval or rejection branches. Approval outcomes in opening a messaging thread between Claimant and Finder; rejection keeps the listing Active.

4. Use Case Diagram

The use case diagram below shows how Claimant and Finder roles interact with the Lost and Found system in Phase 2. A Claimant is an authenticated user who has received a match notification and is responding to it. A Finder is the original poster of a Found listing. Both roles extend the Registered User role from Phase 1, so they retain the ability to browse, search, and manage their own listings in addition to Phase 2 claim and messaging workflows.

Normal case: a claimant receives a match notification, reviews the listing details and private identifying information, and submits evidence in response. The Submit Claim use case includes Validate Claim Evidence and Verify Authentication before evidence is accepted.

Error case 1, invalid evidence: the Display Validation Error use case extends Submit Claim whenever submitted evidence is incomplete or images exceed the configured size limit, so the action is stopped instead of completing.

Error case 2, owner review failure: the Reject Claim use case extends the Review Claim flow whenever the owner rejects evidence, keeping the listing Active for other claimants to respond to.

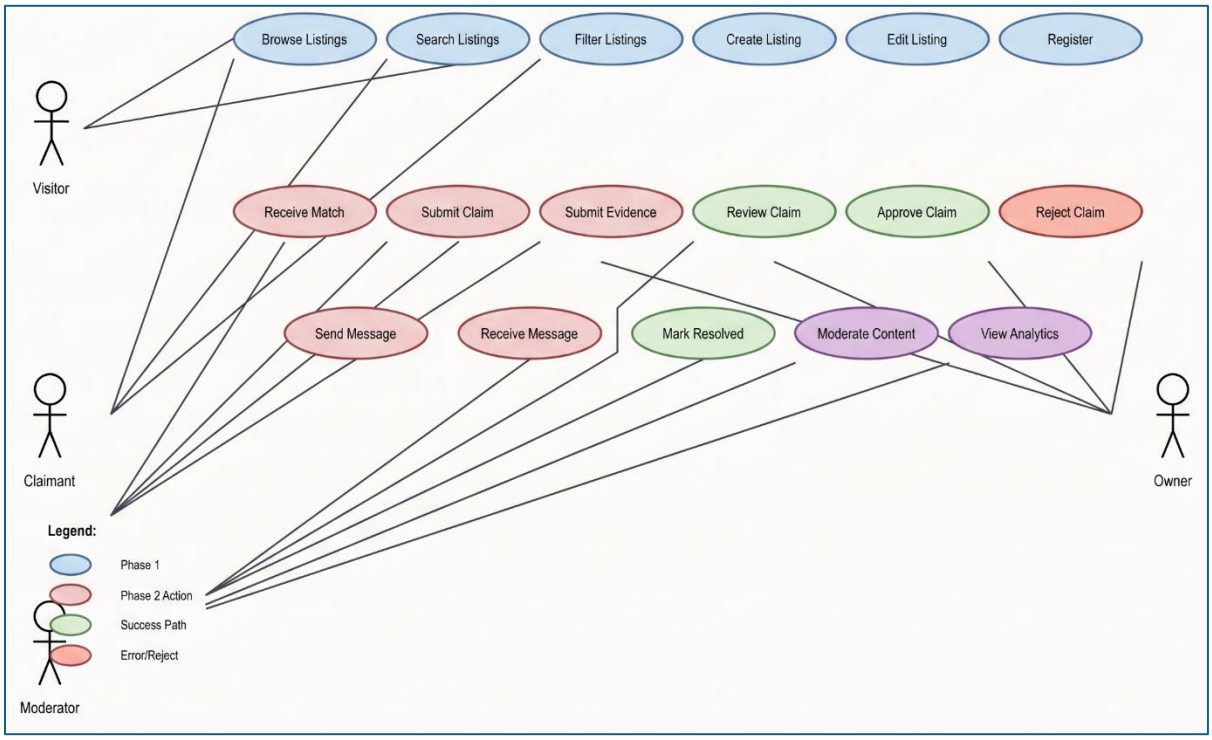


Figure 3. Use case diagram for Phase 2, showing the Claimant and Finder roles, the match notification and claim workflows, successful claim review and approval opening a messaging thread, invalid evidence validation errors, and the claim rejection path.

3. Test Cases (Phase 2)

1. Purpose and Current Status

This plan contains the Deliverable 4 testing work for the Lost and Found for Campus project. The test cases are based on the Phase 2 scope including matching engine, claim verification, notifications, and in-app messaging, defined in Deliverable 2 and detailed in the Deliverable 4 Requirements document.

No Phase 2 test case has been executed yet. Every case is a planned test specification, not a claimed execution result. Test case execution will be completed in the final phase of the project, and actual output, pass or fail status, and defect references will be added then. The backend suite of 83 automated tests is reported separately in the Compilation and Run Instructions.

2. Phase 2 Coverage

The plan covers matching engine scoring and matching algorithm, claim evidence submission and owner review workflows, in-app messaging thread management, notification dispatch and delivery, authorization boundaries for claim and messaging operations, privacy controls on match scores and message contents, and Phase 2 integration paths.

Total planned cases: 104. Every case lists its requirement, what it tests, its preconditions and test action, and its expected output. The complete catalog of all 104 cases appears in Appendix A, and a companion workbook, D4_Item03_Test_Cases_Phase2.xlsx, is included in the project repository so the team can record actual results directly against it as Phase 2 is implemented.

3. Requirement Coverage Summary

The table below maps each Phase 2 requirement area to the full set of test IDs that exercise it, confirming that every functional, nonfunctional, and interface requirement in the Requirements, Phase 2 section has planned test coverage.

Coverage Area	Requirements / Basis	Test IDs / Coverage
Matching engine	FR-MATCH-01 to 03; NFR-PERF-01 to 02	UT-MATCH-01 to 14; INT-01 to 06, 16, 17
Claim verification	FR-CLM-01 to 03; NFR-PRIV-01	UT-CLM-01 to 18; INT-07 to 12, 18 to 20
Notifications	FR-NOTIF-01 to 02; NFR-REL-01 to 02	UT-NOTIF-01 to 12; INT-13 to 15, 21, 22
Messaging	FR-COM-01 to 03; NFR-PRIV-01	UT-MSG-01 to 16; INT-23 to 24
Authorization/privacy	FR-AUTH-03; NFR-PRIV-01	UT-AUTH-01 to 08; UT-PRIV-01 to 12
Integration paths	Phase 2 end to end workflows	INT-01 to 24

4. Test Execution Procedure

Implement backend services (MatchingService, ClaimService, NotificationService, MessagingService) and integration with Express routes after the Phase 1 API foundation is complete.

Implement frontend components (claim forms, message interface, owner review interface) after React Phase 1 is stable.
Execute integration tests after Phase 2 services, routes, and UI components are connected in the test environment.
When test case execution is completed in the final phase of the project, record the actual output for each executed case and set its Status to Pass or Fail.
For a failed case, record the related defect or GitHub issue, apply the fix, and rerun the test before marking it Pass.

Appendix A. Complete Test Case Catalog (104 cases)

Every planned Phase 2 test case, in full. Test case execution will be completed in the final phase of the project.

Test ID	Type	Requirement	What It Tests	Preconditions & Test Action	Expected Output
UT-MATCH-01	Positive	FR-MATCH-01	Score exact category match	Two listings, same category. Action: Run matching engine.	Match score includes perfect category component (1.0).
UT-MATCH-02	Positive	FR-MATCH-01	Score location proximity	Two listings, same building. Action: Run matching engine.	Match score includes location component (>0.8).
UT-MATCH-03	Positive	FR-MATCH-01	Score date proximity	Lost and Found items 1 day apart. Action: Run matching engine.	Match score includes date proximity component (>0.7).
UT-MATCH-04	Positive	FR-MATCH-01	Score keyword overlap	Listings share 3 of 5 description keywords. Action: Run matching engine.	Match score includes keyword component (>=0.6).
UT-MATCH-05	Boundary	FR-MATCH-01	Weighted score combines components	Perfect category + location, poor date + keywords. Action: Calculate combined score.	Final score is weighted combination; no single component dominates.
UT-MATCH-06	Negative	FR-MATCH-01	Category mismatch reduces score	Lost item category A, Found item category B. Action: Run matching engine.	Match score excludes or minimizes category component (<=0.3).
UT-MATCH-07	Negative	FR-MATCH-01	Building mismatch reduces score	Listings in different buildings. Action: Run matching engine.	Match score location component is reduced (<0.4).
UT-MATCH-08	Negative	FR-MATCH-01	Date gap reduces score	Lost and Found items 30 days apart. Action: Run matching engine.	Match score date component is significantly reduced (<0.3).
UT-MATCH-09	Boundary	FR-MATCH-01	No keyword overlap	Listings with completely different description words. Action: Run matching engine.	Match score reflects no keyword match; score is low.
UT-MATCH-10	Performance	NFR-PERF-01	Matching completes within time budget	Dataset with 5,000 listings; new listing posted. Action: Trigger matching engine.	Match results returned within 1 second.
UT-MATCH-11	Performance	NFR-PERF-02	Matching under concurrent load	50 concurrent users; matching engine processing. Action: Run load test.	Matching maintains accuracy under 50 concurrent requests.

UT-MATCH-12	Privacy	NFR-PRIV-01	Match score details excluded from public	Match with detailed score components calculated. Action: Serialize public match response.	scoreDetail (internal components) absent from response.
UT-MATCH-13	Integration	FR-MATCH-01	Matching triggered on new listing creation	Verified user creates Found item; Lost items exist. Action: Submit Found listing.	Matching engine runs automatically; matches queued for notification.
UT-MATCH-14	Negative	FR-MATCH-01	No match if type mismatch	New Lost item; only Found items in database. Action: Run matching engine.	No matches returned; correct behavior for opposite types.
UT-CLM-01	Positive	FR-CLM-01	Claimant submits valid evidence text	Match received; claimant authenticated. Input: description text. Action: Submit claim.	Claim accepted and pending owner review.
UT-CLM-02	Positive	FR-CLM-01	Claimant submits evidence with images	Match received; up to 3 JPEG/PNG images. Action: Submit claim with images.	Claim accepted; images validated and referenced.
UT-CLM-03	Negative	FR-CLM-01	Claimant cannot submit without description	Match received; description field blank. Action: Submit claim.	Submission rejected; description required error shown.
UT-CLM-04	Negative	FR-CLM-01	Claimant submits oversized image	Image >5 MB uploaded with claim. Action: Submit claim.	Image rejected; claim not created; size error returned.
UT-CLM-05	Positive	FR-CLM-02	Owner receives claim for review	Claim submitted by claimant; owner authenticated. Action: Check owner's claim review interface.	Claim displayed for owner review with claimant evidence visible.
UT-CLM-06	Positive	FR-CLM-02	Owner approves matching claim	Owner reviewing claim with matching evidence. Action: Select approve.	Claim marked approved; messaging thread opens; finder notified.
UT-CLM-07	Positive	FR-CLM-02	Owner rejects claim evidence	Owner reviewing claim; evidence does not match private details. Action: Select reject.	Claim marked rejected; listing remains Active for other claimants.
UT-CLM-08	Negative	FR-CLM-02	Non-owner cannot approve claim	Claim pending; User B attempts approval (not owner). Action: Call approve endpoint.	Request denied; claim status unchanged.

UT-CLM-09	Negative	FR-CLM-02	Non-claimant cannot view claim evidence	Claim submitted; User C attempts to view evidence (not claimant/owner). Action: Call claim detail endpoint.	Evidence is excluded from response; unauthorized access blocked.
UT-CLM-10	Privacy	NFR-PRIV-01	Claim evidence excluded from public listing	Claim submitted with identifying details. Action: Serialize public listing.	Claim evidence never included in public response.
UT-CLM-11	Authorization	FR-AUTH-03	Ordinary user cannot access claim admin operations	Authenticated user; admin-only claim report action. Action: Call admin endpoint.	Request denied; no admin state change occurs.
UT-CLM-12	Boundary	FR-CLM-01	Claim with maximum 3 images	Claim with exactly 3 images, each <=5 MB. Action: Submit claim.	All 3 images accepted.
UT-CLM-13	Negative	FR-CLM-01	Claim with 4 images rejected	Claim payload includes 4 image files. Action: Submit claim.	Claim rejected; image count error returned.
UT-CLM-14	Negative	FR-CLM-02	Owner cannot review claim for listing they do not own	Claim pending on listing owned by User A; User B (ordinary user). Action: Attempt claim review.	Authorization check blocks review; User B cannot approve/reject.
UT-CLM-15	Integration	FR-CLM-01 + FR-CLM-02	Claim evidence safely stored and retrieved	Claimant submits claim with text and images; owner retrieves it. Action: Submit claim and owner opens claim detail.	Claim retrieval returns exact evidence submitted.
UT-CLM-16	Reliability	NFR-REL-01	Claim approval confirmed before marking resolved	Owner approves claim; database write mocked to fail. Action: Submit approval.	Failure returned; listing not marked resolved.
UT-CLM-17	Negative	FR-CLM-02	Cannot approve same claim twice	Claim already approved; owner attempts second approval. Action: Call approve endpoint again.	Second approval rejected; claim remains in approved state.
UT-CLM-18	Usability	FR-CLM-01	Validation error preserves valid claim fields	Claim form with invalid image, valid description. Action: Submit and correct image.	Valid description text remains populated after error.
UT-NOTIF-01	Positive	FR-NOTIF-01	In-app notification created on match found	Match generated between listings. Action: Trigger matching engine.	Notification created and queued for delivery.

UT-NOTIF-02	Positive	FR-NOTIF-01	Email notification sent on match	Match generated; notification service triggered. Action: Verify email delivery.	Email message sent to both finder and claimant with match details.
UT-NOTIF-03	Positive	FR-NOTIF-02	In-app notification on claim submission	Claimant submits evidence. Action: Monitor notification service.	In-app notification created for listing owner.
UT-NOTIF-04	Positive	FR-NOTIF-02	Email notification on claim submission	Claimant submits evidence. Action: Check email delivery.	Email sent to listing owner with claim details.
UT-NOTIF-05	Positive	FR-NOTIF-02	Email sent on claim approval	Owner approves claim. Action: Verify email.	Email sent to claimant confirming approval and messaging channel opening.
UT-NOTIF-06	Positive	FR-NOTIF-02	Email sent on claim rejection	Owner rejects claim. Action: Verify email.	Email sent to claimant explaining rejection.
UT-NOTIF-07	Negative	FR-NOTIF-01	User can opt out of email notifications	User preference set to email off. Match found. Action: Verify email not sent.	No email sent; in-app notification only.
UT-NOTIF-08	Reliability	NFR-REL-01	Notification not marked sent before delivery succeeds	Email service failure mocked. Action: Attempt notification send.	Notification remains Pending; not marked SentEmail.
UT-NOTIF-09	Reliability	NFR-REL-02	Failed notification can be retried	Notification marked Failed. Action: Trigger retry process.	Notification reprocessed and delivery attempted again.
UT-NOTIF-10	Privacy	NFR-PRIV-01	Notification excludes sensitive fields	Notification created with match and claim data. Action: Serialize notification body.	Passwords, tokens, internal scores excluded; user-facing data only.
UT-NOTIF-11	Integration	FR-NOTIF-01 + FR-NOTIF-02	Notification chain: match to claim to messaging	Match found, claim submitted, approval triggered. Action: Monitor notification service.	Three separate notifications created and sent in sequence.
UT-NOTIF-12	Boundary	FR-NOTIF-01	Rate limit on notifications	Many matches queued in short time. Action: Trigger high volume matching.	Notification queue respects batch/rate limits; no overwhelming delivery.
UT-MSG-01	Positive	FR-COM-01	Message thread created on claim approval	Owner approves claim. Action: Check messaging interface.	Message thread opened between finder and claimant.

UT-MSG-02	Positive	FR-COM-01	Claimant sends first message	Thread opened; claimant authenticated. Input: message text. Action: Submit message.	Message stored and visible to both participants.
UT-MSG-03	Positive	FR-COM-01	Finder responds to claimant	Thread exists; claimant sent message; finder authenticated. Input: response text. Action: Submit message.	Response message stored; claimant sees new message.
UT-MSG-04	Negative	FR-COM-01	Third party cannot access message thread	Thread exists between finder and claimant; User C attempts read. Action: Call message endpoint.	Authorization denies access; messages not returned.
UT-MSG-05	Negative	FR-COM-01	Third party cannot send messages	Thread exists; User C attempts to send message. Action: Call message send endpoint.	Request denied; message not stored.
UT-MSG-06	Negative	FR-COM-01	Empty message rejected	Thread exists; empty message text. Action: Submit message.	Submission rejected; message required error returned.
UT-MSG-07	Privacy	NFR-PRIV-01	Message contents never exposed in public responses	Messages exchanged in thread. Action: Retrieve public listing.	Message contents never appear in public API responses.
UT-MSG-08	Privacy	NFR-PRIV-01	Message thread hidden from non-participants	Thread exists; non-participant user. Action: Call list message threads endpoint.	Thread not listed for unauthorized user; access blocked.
UT-MSG-09	Positive	FR-COM-02	Moderator can view all messages	Messages exchanged; moderator authenticated. Action: Access message thread as moderator.	Moderator sees full conversation and can monitor.
UT-MSG-10	Positive	FR-COM-02	Moderator can flag problematic messages	Message contains concerning content; moderator authenticated. Action: Flag message.	Message flagged and logged; both participants notified.
UT-MSG-11	Boundary	FR-COM-01	Long message accepted and stored	Message text 1000 characters. Action: Submit message.	Full message text stored and retrieved unchanged.
UT-MSG-12	Negative	FR-COM-03	Message with script-like content handled as text	Message contains HTML/script tags. Action: Submit and retrieve message.	Message stored as plain text; tags treated as text, not executed.

UT-MSG-13	Usability	FR-COM-01	Message timestamp recorded	Message sent. Action: Retrieve message with metadata.	Message includes timestamp of creation.
UT-MSG-14	Integration	FR-COM-01 + FR-COM-02	Message thread reflects approval and claim context	Thread opened via claim approval; both participants can reference claim. Action: View thread details.	Thread shows associated claim ID and approval details.
UT-MSG-15	Reliability	NFR-REL-01	Message persisted before confirmation	Message submitted; database write forced to fail. Action: Submit message.	Failure returned; message not reported as sent.
UT-MSG-16	Privacy	NFR-PRIV-01	Email of message participants never exposed in messages	Messages exchanged. Action: Inspect message text and metadata.	Email addresses absent from message content and headers.
UT-AUTH-01	Negative	FR-AUTH-03	Unverified user cannot submit claim	Registered unverified account; claim submission attempted. Action: Call claim endpoint.	Claim request denied; unverified status enforced.
UT-AUTH-02	Negative	FR-AUTH-03	Non-finder cannot review claim on found listing	Listing owned by User A; User B attempts to review claim. Action: Call claim review endpoint.	Authorization denies review; ownership check enforced.
UT-AUTH-03	Negative	FR-AUTH-03	Non-claimant cannot send messages in thread	Thread established; User C attempts message send. Action: Call message send endpoint.	Authorization blocks send; User C not recognized as thread participant.
UT-AUTH-04	Positive	FR-AUTH-03	Finder can review all claims on their listing	Listing owned by User A; multiple claims submitted. Action: User A calls claim review endpoint.	All pending claims listed for owner review.
UT-AUTH-05	Positive	FR-AUTH-03	Claimant can message only in approved threads	Thread opened via approved claim. Claimant authenticated. Action: Send message.	Message accepted and stored.
UT-AUTH-06	Negative	FR-AUTH-03	Admin only operations blocked for ordinary users	Ordinary user authenticated. Admin-only claim report action. Action: Call admin endpoint.	Request denied; no admin capability conferred.
UT-AUTH-07	Positive	FR-AUTH-03	Moderator can monitor all messages	Moderator authenticated. Action: List all message threads.	All active threads displayed for monitoring.

UT-AUTH-08	Boundary	FR-AUTH-03	Role check enforced at API even if UI omits control	Ordinary user direct API call to claim approve (UI button hidden). Action: Call API endpoint directly.	API enforces ownership authorization; approval fails.
UT-PRIV-01	Privacy	NFR-PRIV-01	Private listing details excluded from match notification	Found listing with private details; match notification sent. Action: Check notification content.	Private details absent from notification message.
UT-PRIV-02	Privacy	NFR-PRIV-01	Match score details excluded from public serialization	Match with component scores calculated. Action: Serialize for public API.	scoreDetail object absent from response.
UT-PRIV-03	Privacy	NFR-PRIV-01	Claim evidence excluded from non-owner view	Claim submitted with evidence. User C (not owner/claimant) calls claim endpoint. Action: Retrieve claim.	Evidence fields absent from response.
UT-PRIV-04	Privacy	NFR-PRIV-01	Message contents never logged in plain text	Messages exchanged. Action: Inspect server logs.	Message text not present in application logs.
UT-PRIV-05	Privacy	NFR-PRIV-01	Email of non-owner never exposed in claim context	Claim submitted. Unauthorized user views claim. Action: Call claim endpoint.	Finder/claimant email addresses absent from unauthorized response.
UT-PRIV-06	Privacy	NFR-PRIV-01	Claimant email hidden from finder message thread	Thread established; message sent. Finder attempts to extract claimant email. Action: Inspect thread metadata.	Claimant email not included in any thread field.
UT-PRIV-07	Privacy	NFR-PRIV-01	Match algorithm weights/scores excluded from non-admin	Match with internal weight details. Ordinary user attempts to view. Action: Call match endpoint.	Detailed scoring method and weights absent from response.
UT-PRIV-08	Privacy	NFR-PRIV-01	Private details never cached in public search results	Found listing with private details; search performed. Action: Search and inspect results.	Private details absent from all matching results in list.
UT-PRIV-09	Privacy	NFR-PRIV-01	Notification content excludes claim evidence	Claim notification sent to owner. Action: Inspect notification message.	Claimant evidence summary absent; owner directed to review in system.

UT-PRIV-10	Privacy	NFR-PRIV-01	Moderator flagged message excluded from participant notifications	Moderator flags message; participants notified. Action: Check notification sent to participants.	Flag details (moderation action, reason) excluded from user notification.
UT-PRIV-11	Privacy	NFR-PRIV-01	Score calculations and intermediate values not logged	Match scoring process. Action: Inspect debug/application logs.	Component scores and weight application absent from logs.
UT-PRIV-12	Privacy	NFR-PRIV-01	Error messages do not expose internal state	Matching or claim operation error triggered. Action: Inspect error response.	Error message is user facing; internal values not leaked.
INT-01	Integration	FR-MATCH-01 + FR-MATCH-02	Match engine to notification flow	New listing posted matching existing item. Action: Monitor notification service.	Matching runs; notifications queued; emails sent.
INT-02	Integration	FR-MATCH-01 + FR-CLM-01	Match notification to claim submission	Notification received by claimant; claim form opened. Action: Submit claim.	Claim accepted and stored; owner receives review notification.
INT-03	Integration	FR-CLM-02 + FR-COM-01	Claim approval to messaging channel	Owner approves claim. Action: Check for opened thread.	Message thread created; both parties can send messages.
INT-04	Integration	FR-CLM-02 + FR-NOTIF-02	Claim approval notification and messaging chain	Owner approves claim. Action: Monitor notifications and threads.	Notification sent to claimant; thread opens simultaneously.
INT-05	Integration	FR-NOTIF-01 + FR-NOTIF-02	Match notification includes call to action	Match notification sent. Action: Inspect email/in-app message.	Notification contains claim submission link/button.
INT-06	Integration	FR-MATCH-01 + FR-CLM-01 + FR-COM-01	Full claim flow from match to messaging	Match found; claimant submits evidence; owner approves; messages exchanged. Action: Execute full workflow.	All steps complete; final message stored; listing marked resolved.
INT-07	Integration	FR-CLM-02 + FR-NOTIF-02	Rejection notification sent to claimant	Owner rejects claim. Action: Monitor notifications.	Email/in-app notification sent to claimant explaining rejection.

INT-08	Integration	FR-CLM-01 + NFR-PRIV-01	Claim evidence privacy across API and storage	Claimant submits evidence with sensitive details. Owner retrieves. Unauthorized user attempts retrieval. Action: Observe evidence in each case.	Owner sees full evidence; unauthorized user sees none.
INT-09	Integration	FR-MATCH-01 + NFR-PRIV-01	Match privacy across notification and public API	Match created between listings with private details. Action: Check notification, public listing, and internal storage.	Notification excludes private details; public listing excludes private details; internal storage preserves both.
INT-10	Integration	FR-COM-01 + NFR-PRIV-01	Message thread privacy enforcement	Thread established. Participants and non-participants access attempted. Action: Authorized access then unauthorized.	Participants see all messages; non-participants get access denied.
INT-11	Integration	FR-MATCH-01 + FR-CLM-01 + FR-AUTH-03	Authorization across matching and claiming	Match generated; unverified claimant attempts claim; verified claimant submits claim. Action: Call endpoints.	Unverified claim denied; verified claim accepted.
INT-12	Integration	FR-COM-01 + FR-AUTH-03	Message authorization across listing participants	Thread established between finder and claimant. Third party attempts read/write. Action: API calls.	Third party reads and writes denied; participants allowed.
INT-13	Integration	FR-NOTIF-01 + FR-NOTIF-02	Multiple matching notifications do not overwhelm	Claimant receives 5 match notifications in rapid succession. Action: Monitor notification service.	All notifications queued and delivered without loss or duplication.
INT-14	Integration	FR-MATCH-01 + FR-CLM-01	Multiple claimants can submit evidence for same match	Match generated; Claimant A submits; Claimant B submits. Action: Owner reviews both claims.	Both claims stored; owner can review and approve/reject independently.
INT-15	Integration	FR-CLM-02 + FR-COM-01	Multiple messages in single thread	Thread opened; 10 messages exchanged. Action: Retrieve full thread history.	All 10 messages retrieved in correct order without loss.
INT-16	Integration	FR-MATCH-01 + NFR-PERF-01	Matching performance under realistic data load	5000 listings in database; new listing triggers matching. Action: Measure completion time.	Matching completes within 1 second; no UI blocking.

INT-17	Integration	FR-MATCH-01 + NFR-PERF-02	Notification delivery under concurrent match load	50 concurrent users; 100 matches per second generated. Action: Monitor notification throughput.	All notifications queued without loss; delivery rate sustained.
INT-18	Integration	FR-CLM-01 + NFR-USA-01	Claim form validation and error display	Claim form with invalid image and missing description. Action: Submit and correct each field.	Specific errors shown for each field; valid values preserved.
INT-19	Integration	FR-COM-01 + NFR-USA-01	Message composition interface usability	Message form rendered; user types long message. Action: Send message.	Form accepts input without lag; send succeeds; message appears in thread.
INT-20	Integration	FR-CLM-02 + NFR-REL-01	Claim approval transaction safety	Owner approves claim; database fails mid transaction. Action: Monitor rollback.	Approval not reported; claim remains Pending; no orphan messaging thread.
INT-21	Integration	FR-COM-01 + NFR-REL-01	Message send transaction safety	Participant sends message; database write fails. Action: Verify state.	Message not stored; not reported as sent to either party.
INT-22	Integration	FR-NOTIF-01 + NFR-REL-02	Notification retry after transient failure	Email service temporarily unavailable; notification queued. Action: Wait and monitor retry.	Notification reprocessed after service recovery; delivered successfully.
INT-23	Integration	FR-COM-02 + FR-AUTH-03	Moderator monitoring of claims and messages	Moderator authenticated; multiple threads and claims exist. Action: Access monitoring views.	Moderator can list and view all claims/messages; authorize flag action.
INT-24	Integration	FR-MATCH-01 + FR-CLM-01 + FR-COM-01 + FR-NOTIF-02	Complete end-to-end Phase 2 workflow with all subsystems	New listing posted; match found; claim submitted; approved; message thread established; notifications sent. Action: Execute complete workflow.	All steps succeed; notifications sent; messages stored; listing marked resolved.

4. User Manual (Phase 2)

Every screen shown in this manual is a faithful, pixel accurate recreation built directly from the project's real page components and stylesheet, reproducing the exact labels, fields, buttons, and messages the application uses. They were generated this way because a live, running instance was not reachable from the environment this manual was prepared in; they are not AI generated impressions of a hypothetical design.

1. Introduction

Lost and Found for Campus is a web application that lets students, faculty, and staff report lost or found items, search existing listings, and manage the listings they have posted themselves. Phase 2 adds automated matching: when you post a listing, the system finds similar items already posted and notifies both you and the person who found (or lost) the match. You can then submit identifying details called evidence to claim a match, the owner reviews your claim, and if approved, a secure in-app messaging channel opens so you can coordinate the handoff.

This manual covers all Phase 2 features: everything from Phase 1 (creating an account, verifying your email, signing in, resetting a forgotten password, browsing and searching listings, creating a Lost or Found listing with photos, and editing or removing your own listings), plus the new Phase 2 workflows (receiving and understanding match notifications, submitting claim evidence, reviewing claims you have received, and using the in-app messaging system to coordinate handoffs).

This manual is written for the people who will use the application day to day. If you are looking for instructions on installing dependencies and running the project from source code, see the separate Compilation and Run Instructions document.

2. Accessing the Application

Open a supported web browser (current Chrome, Edge, Firefox, or Safari) on a desktop, laptop, tablet, or phone. No specialized hardware, plugin, or app installation is required.

During development and grading, the application runs locally. Your instructor or team will provide the address to open; by default this is http://localhost:5173 for the site itself. You do not need to know or type any backend or database address; the page you open talks to the server automatically.

3. Creating an Account

Select Register from the top navigation bar. You will need:

- Your first and last name.
- A campus email address ending in @unt.edu or @my.unt.edu. Any other email address is rejected with an on-screen message.
- A password of at least 8 characters.

After you submit the form, your account is created immediately, but it is not yet verified. The application automatically sends a verification email to the address you registered with.

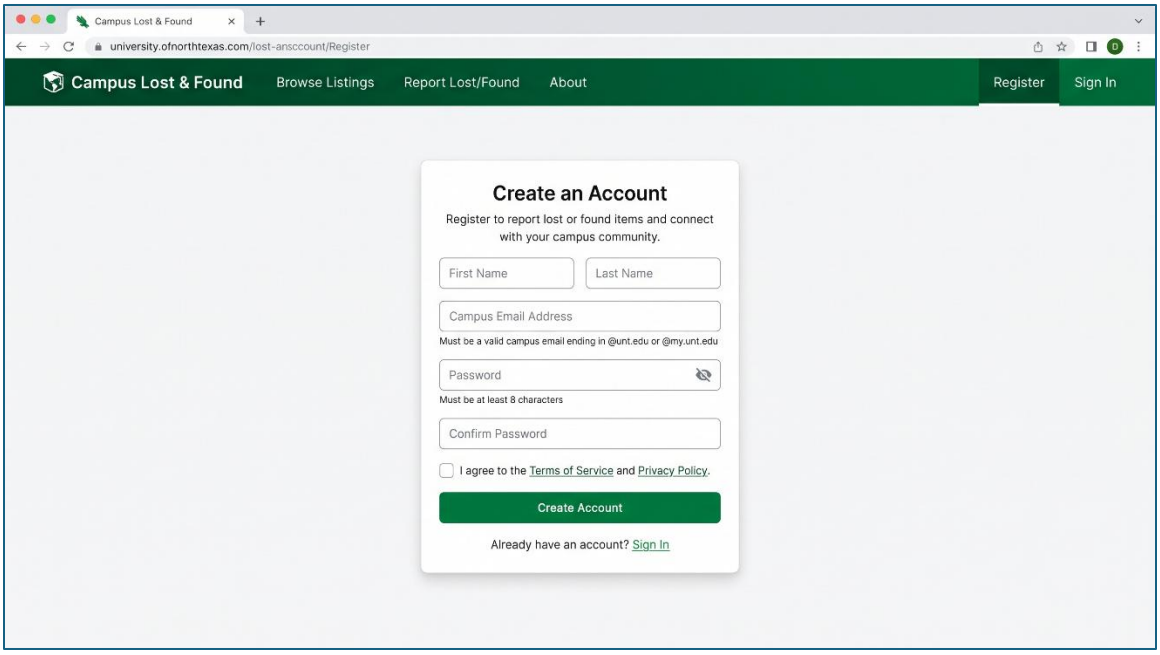


Figure 1. Register page

4. Verifying Your Email

Open the verification email and select its link. The link is valid for 24 hours. Opening it completes verification immediately and shows a confirmation screen; you can then sign in.

You will not be able to create a listing until your email is verified. If you try beforehand, the application tells you so directly rather than failing silently.

If the link has expired or was already used, the screen tells you the link is invalid or expired. During development, if no email provider has been configured yet, the verification link is printed to the backend server's console instead of being emailed; ask whoever is running the server for the link in that case.

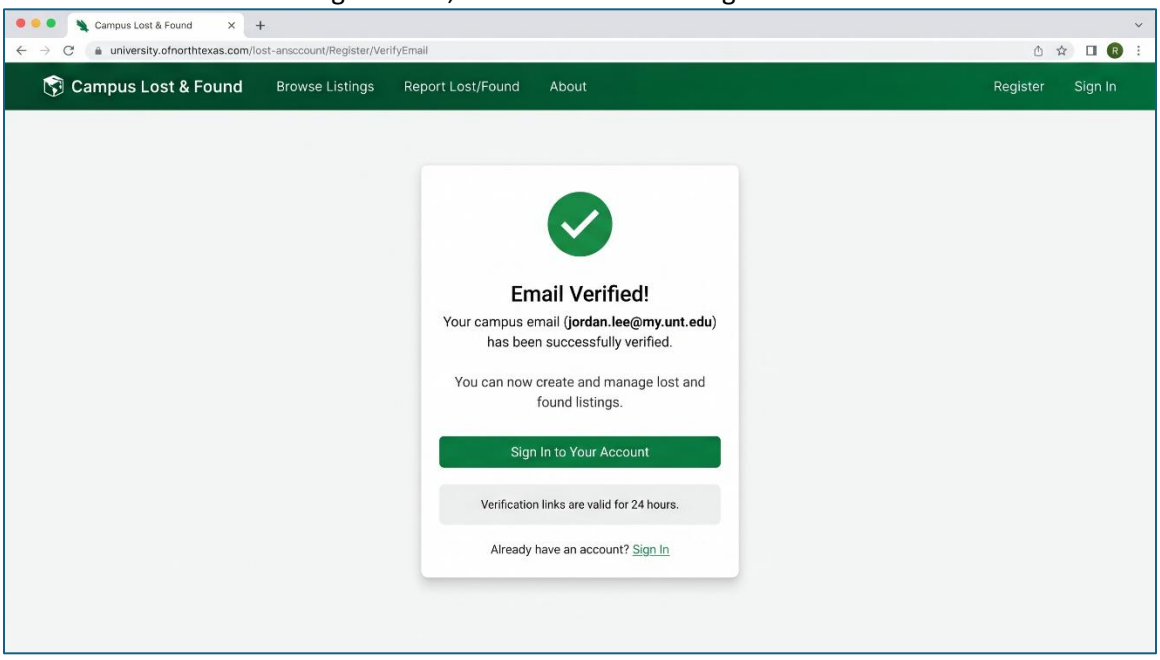


Figure 2. Email verification confirmation screen

5. Signing In and Out

Select Login, enter your email and password, and submit. A successful sign in takes you to the Listings page. If your credentials are not recognized, the application shows a single, generic "Invalid email or password" message; it does not reveal whether the email or the password was the problem, which is a deliberate privacy protection.

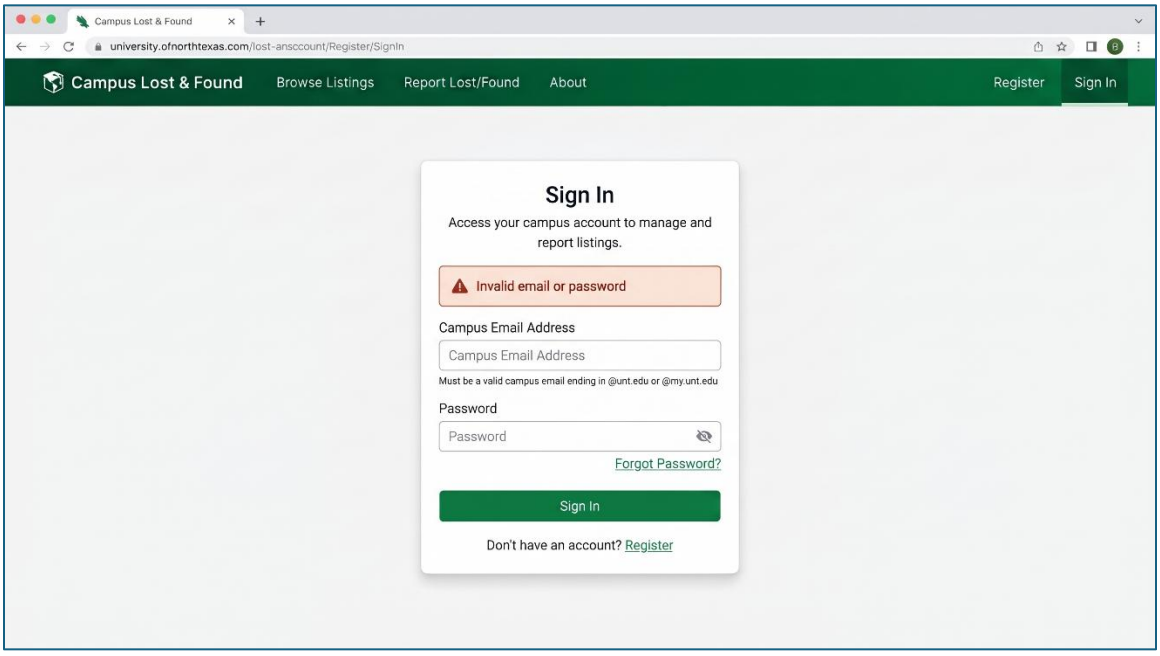


Figure 3. Login page

Once signed in, the navigation bar shows a Create Listing link, a Notifications link, and a Logout button in place of Login and Register. Selecting Logout immediately signs you out on this device; you will need to sign in again to create, edit, or delete a listing.

6. Resetting a Forgotten Password

If you cannot remember your password, select Forgot your password? on the Login page, enter your email, and submit. For your privacy, the application shows the same confirmation message whether or not that email actually has an account; this prevents anyone from using this form to discover who is registered.

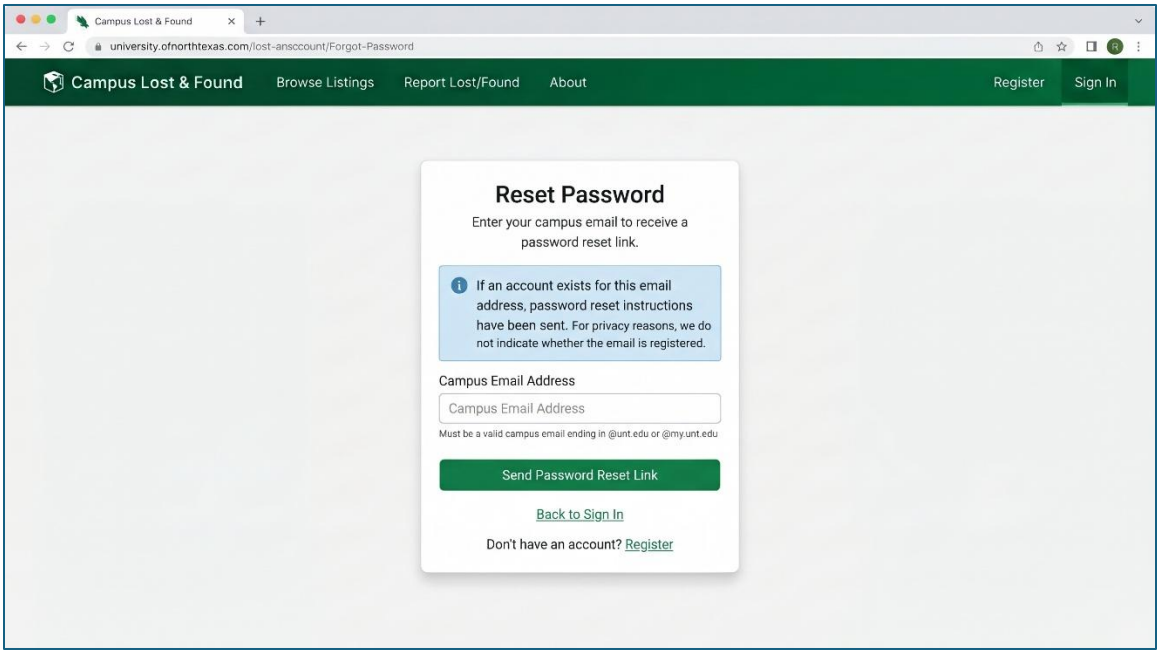


Figure 4. Forgot Password confirmation

If an account exists, a reset email is sent with a link valid for 1 hour. Opening it brings you to a page where you set a new password, at least 8 characters, entered twice to confirm it was typed correctly.

7. Browsing and Searching Listings

Select Listings from the navigation bar. You do not need to sign in to browse or search; this page is open to any visitor. Only Active listings (ones that have not been resolved or removed by their owner) are shown.

You can combine any of the following at once:

- Search: matches your keywords against each listing's title, description, and category.
- Type: Lost or Found.
- Category: for example Bags, Electronics, Keys.
- Building and Zone: the campus location where the item was lost or found.
- Date From / Date To: restricts results to a date range.

Results are shown 20 at a time, with Previous and Next controls at the bottom of the page. Selecting Clear resets every filter and returns to the first page.

If you are signed in and a listing is one you posted yourself, Edit and Delete buttons appear directly on that listing's card; they do not appear on listings posted by other people.

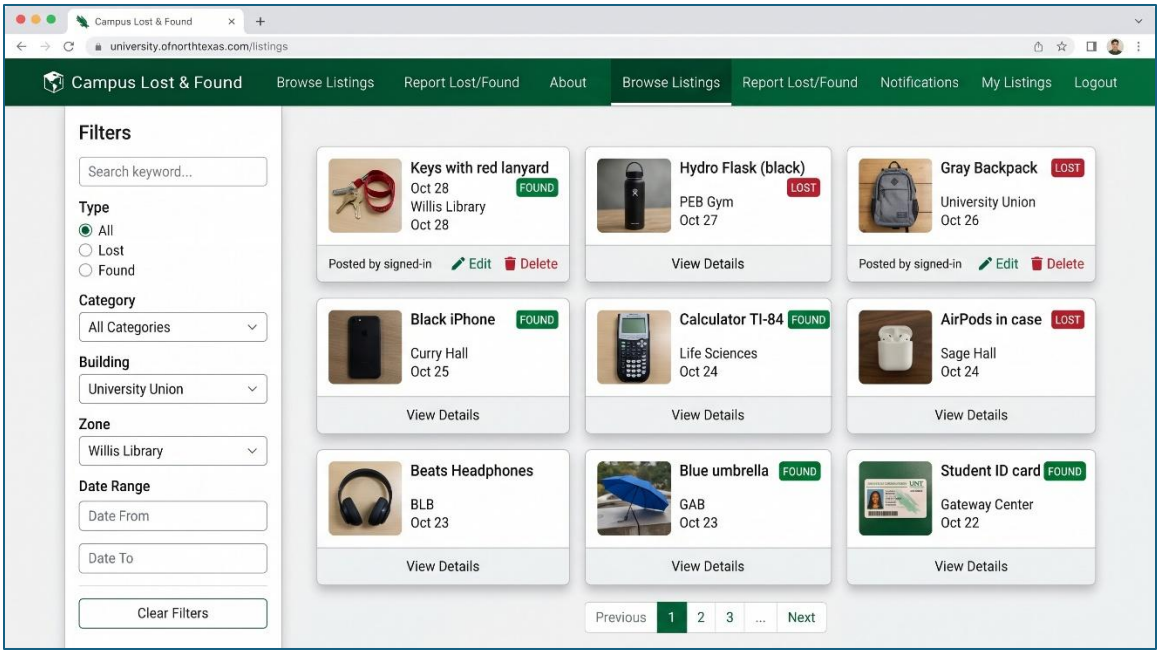


Figure 6. Listings page with search and filter controls

8. Creating a Lost or Found Listing

You must be signed in and have a verified email to create a listing. Select Create Listing and complete the form:

i. Type: choose Lost or Found.
ii. Title, Category, Description, and Date: all required.
iii. Building: required. Zone is optional and narrows the location further.
iv. Photos: choose up to 3 image files (JPEG, PNG, or WebP, 5 MB or smaller each). A Found listing requires at least one photo; a photo is optional for a Lost listing, since you may not have one of an item you lost.
v. Private Identifying Details: optional, for Found listings. Describe something only the true owner would know (an engraving, a sticker, contents of a bag). This is never shown on the public listing, in search results, or to anyone other than you; it exists so you can verify a claimant's identity later during the claim review step.

Submitting the form with a required field missing does not clear anything you already entered; the specific missing or invalid field is identified on screen so you can fix just that one field.

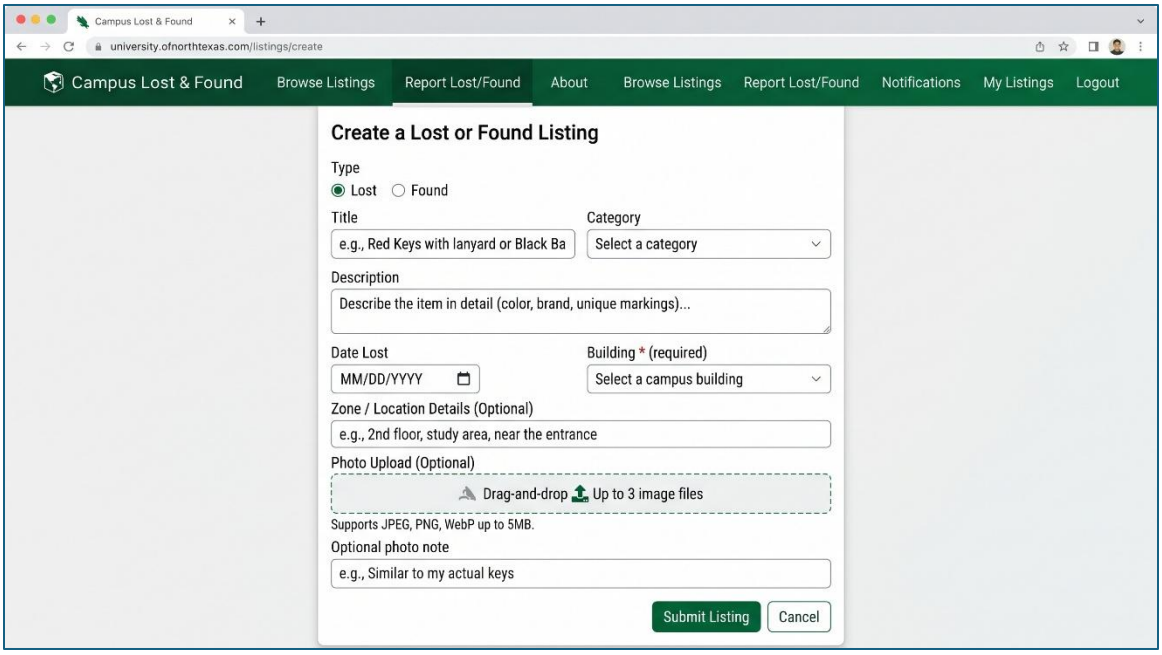


Figure 7. Create Listing page

9. Editing Your Own Listing

From the Listings page, select Edit on any listing you posted. The form opens pre-filled with that listing's current information. Change whatever you need and select Save Changes.

Photos are optional on this form: leave the photo field empty to keep the listing's existing photos exactly as they are, or choose new files to replace all of them at once.

Only the original poster can edit a listing. If you attempt to edit a listing you do not own (for example, by opening its edit link directly), the application rejects the request.

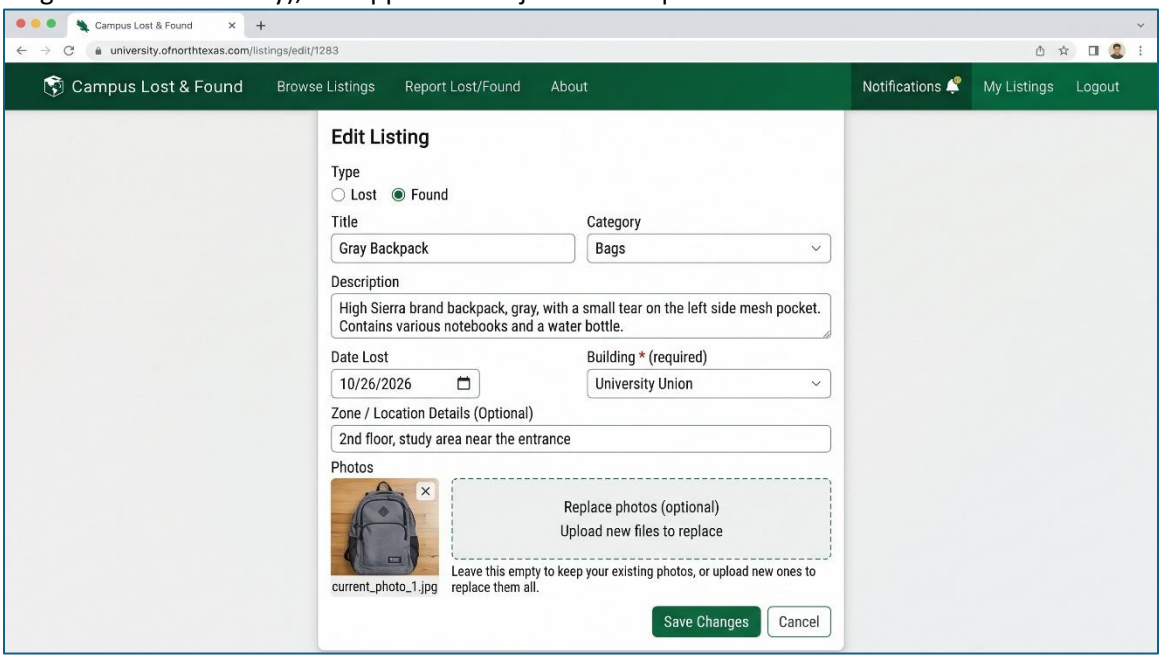


Figure 8. Edit Listing page

10. Removing Your Own Listing

Select Delete on a listing you posted. You will be asked to confirm before anything happens. Deleting a listing removes it from the public Listings page immediately; the record itself is kept rather than erased, consistent with how the course project is designed, but you will no longer see or be able to recover it from the application. As with editing, only the original poster can delete a listing.

11. Understanding Notifications

When you post a listing, the matching engine searches existing listings for similar items. If a likely match is found, you receive a match notification; the person who posted the matching listing also receives a notification, and they will see a claim submission form if they want to claim your item.

From the navigation bar, select Notifications to see all pending match notifications and claim activity. You will also receive in-app alerts (notifications shown at the top of the page) and email notifications when:

- A potential match is found for one of your listings.
- Someone submits a claim to one of your listings (you see this as a claim to review).
- A claim you submitted is approved or rejected.
- Someone sends you a message in an active message thread.

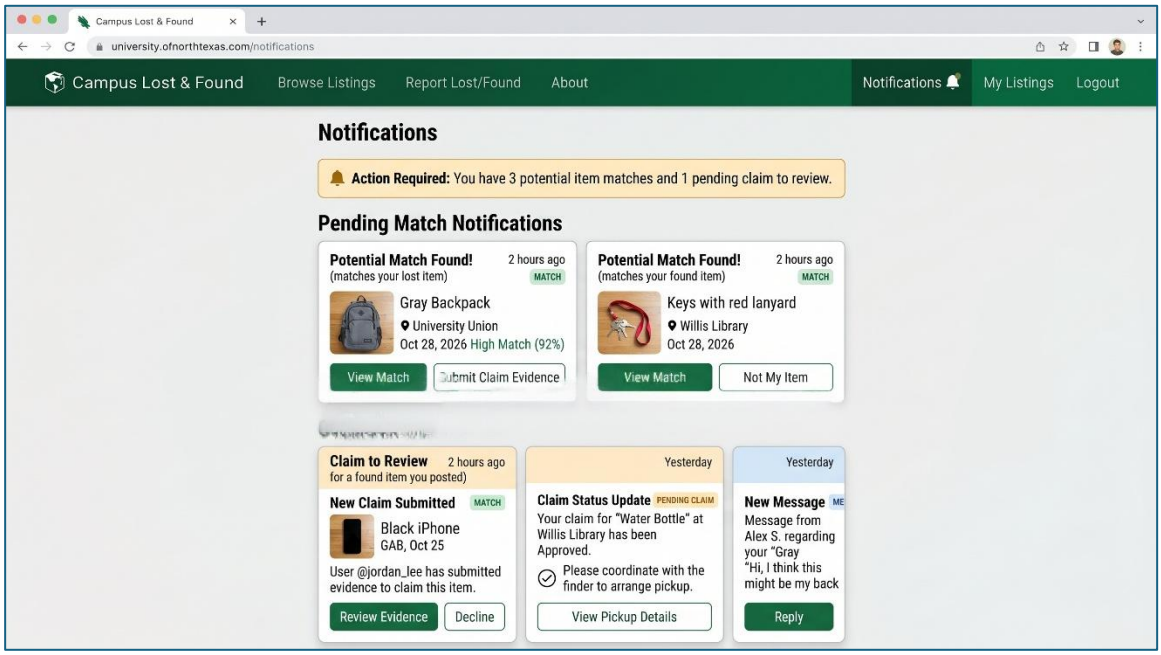


Figure 9. Notifications page

12. Responding to a Match Notification

When the matching engine finds a potential match for your listing, you receive a notification with the matching listing's title, location, and primary photo. You can review the match details from either the Notifications page or from the badge on the listing itself.

Selecting Review Match shows both listings side by side so you can compare them. The matching listing's public details are shown, but not the private identifying details (those are only revealed if the matching person submits a claim). A score from 0 to 1.0 appears, indicating the engine's confidence that these are the same item; higher scores mean the engine found more similarities.

From this screen you can:

- Accept the match and wait for a claim. The system stays ready to receive claim evidence if the other person decides to submit one.
- Reject the match if you are certain it is not the same item. Rejected matches do not appear again; you can re-enable matching for this pair of listings if you change your mind.
- Go back without making a decision and review it later.

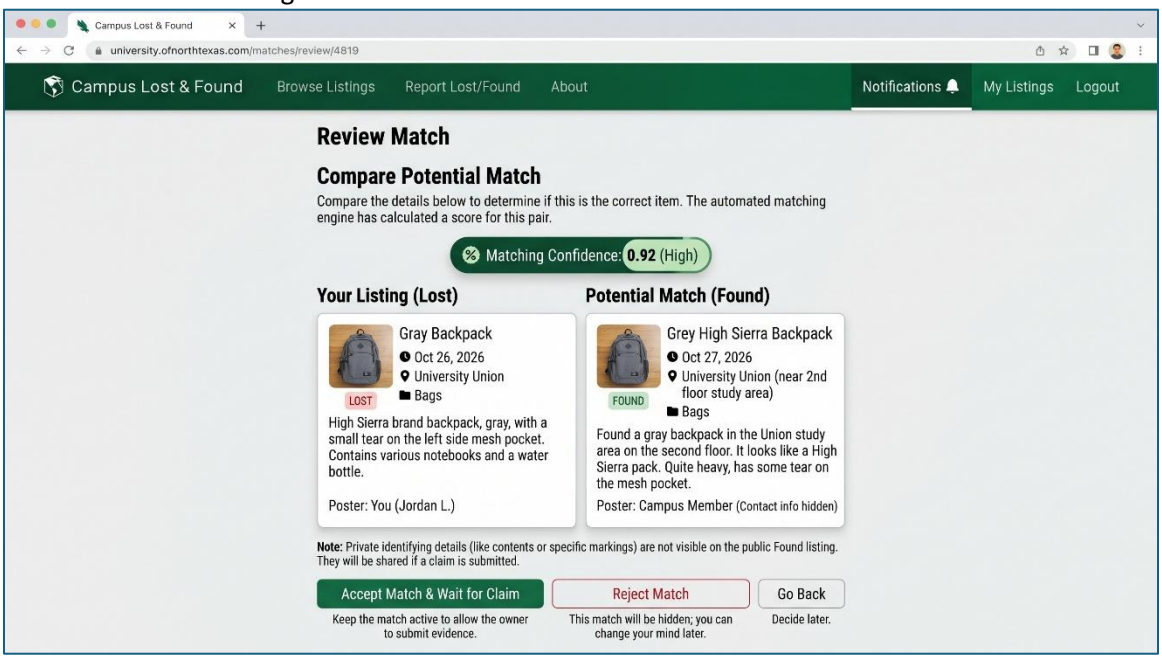


Figure 10. Match review screen

13. Submitting Claim Evidence

If you see a match notification and you believe you have found (or lost) the matching item, you can submit a claim. Select Submit Claim from the match notification or match review screen.

A claim form opens where you provide:

vi. Evidence Description: required. Describe identifying details that prove you own or found the item. For a Lost item you are claiming, describe details from your own experience. For a Found item you are claiming to own, describe details that match the private identifying details the finder stored.

vii. Evidence Photos: optional. You may submit up to 2 photos of the item you found, or yourself with the item, to strengthen your claim.

Submit the form and the claim is sent to the owner of the matching listing. They will see a notification that a claim has been submitted and can review your evidence. You will see a confirmation message and the claim moves to Pending status.

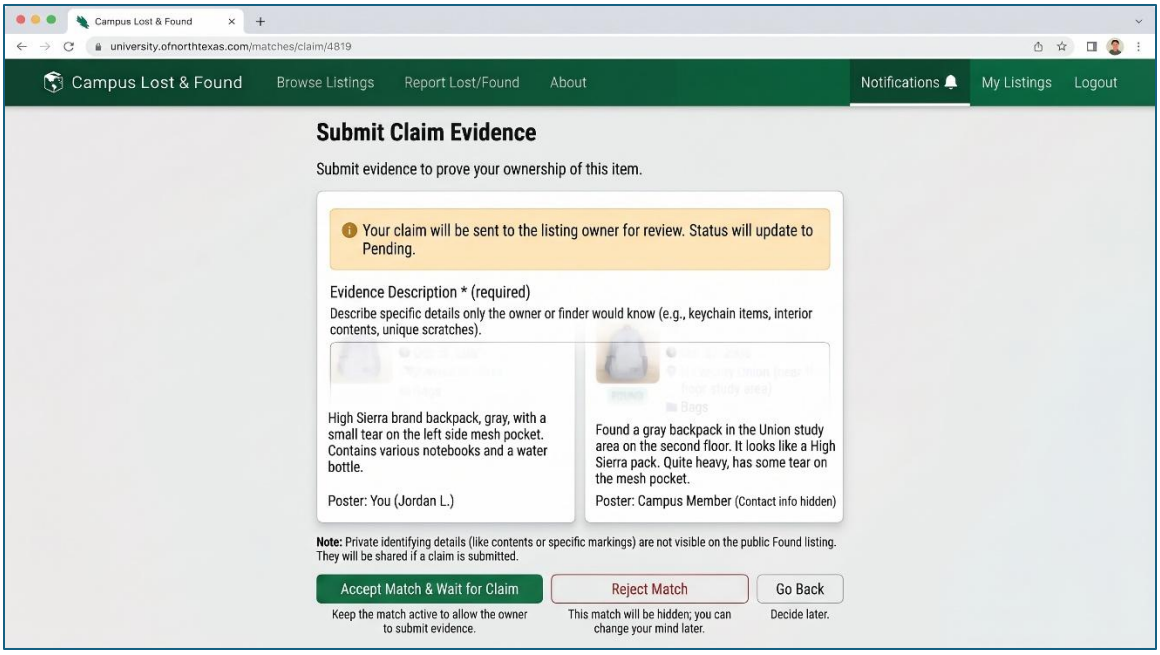


Figure 11. Submit Claim form

14. Reviewing and Approving Claims

When someone submits a claim to one of your listings, you receive a claim review notification. The notification includes the claimant's name and the item they are claiming.

Select Review Claim from the notification to see the full claim details. The claim shows:

- The claimant's evidence description.
- Any photos the claimant submitted as evidence.
- Your own private identifying details from when you created the listing, shown side by side with their claim so you can verify the details match.

The matching listing's details are shown for reference.

You can then:

- Approve the claim if the evidence matches your private details and you believe they are the true owner or finder. Approving opens an in-app message thread between you and the claimant so you can coordinate the handoff.
- Reject the claim if the evidence does not match your private details or you are not confident they are the right person. Rejecting keeps your listing Active so other claimants can submit evidence; you can change your mind and review the same claim again later.

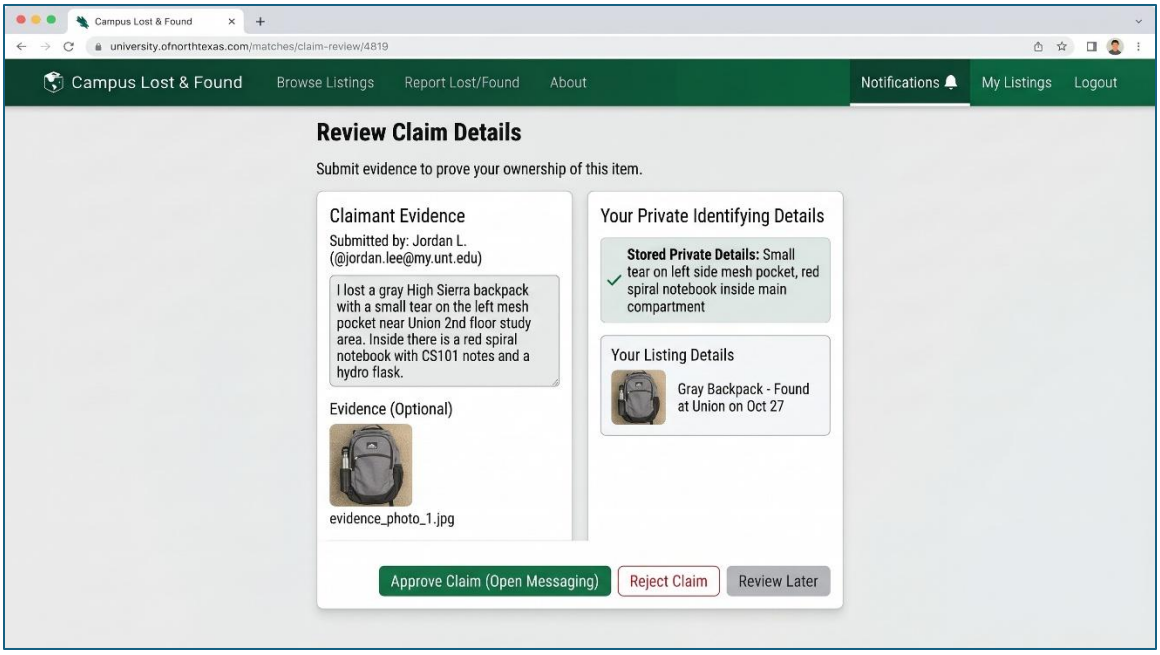


Figure 12. Claim review screen

15. Using In-App Messaging

Once you approve a claim, an in-app message thread opens automatically between you (the listing owner/finder) and the claimant. The thread is private to the two of you. Neither party's personal contact information such as phone numbers or personal email addresses is exposed in the messages; all communication happens through this secure channel.

From the Listings page or Notifications page, select Open Message or open the conversation from your notifications. The message interface shows the conversation history at the top and a text input field at the bottom.

Type your message, review it, and select Send. Messages are delivered immediately and both parties receive an in-app notification and an optional email notification that a new message has arrived. Read receipts are not shown; once you send a message, you can assume it was delivered.

When you and the other party have completed the handoff (either in person or through the messaging channel), the listing owner should delete or mark the listing as Resolved from the Listings page, which closes the match and the message thread.

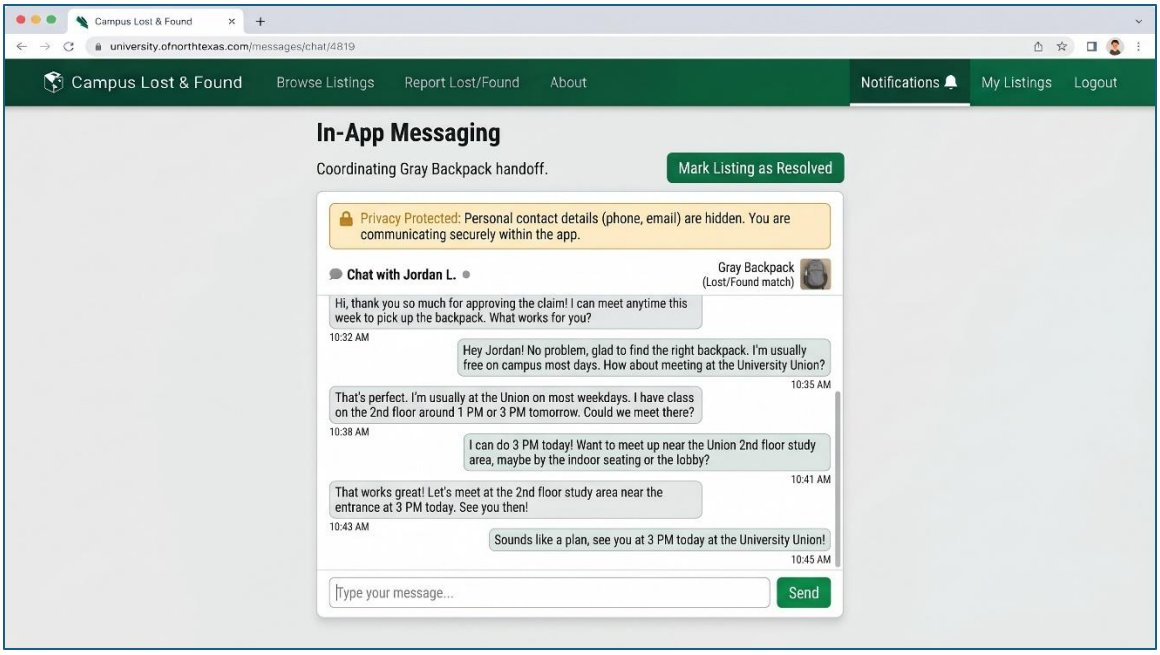


Figure 13. In-app messaging screen

16. Understanding On-Screen Messages

The application is designed to tell you specifically what went wrong rather than failing silently. The table below lists messages you are likely to encounter in Phase 2 and what to do about it.

Situation	What You See	What To Do
A required field is left blank or invalid	The field is outlined and a specific message appears directly beneath it (for example, a UNT email address is required).	Correct the field indicated and submit again. Every other field you already filled in stays as you left it.
Registering with a non UNT email	"Please use a valid UNT email address."	Use an address ending in @unt.edu or @my.unt.edu.
Signing in with the wrong email or password	"Invalid email or password."	re-enter your credentials, or use Forgot your password? if you are not sure.
Creating a listing before verifying your email	"Please verify your email before creating a listing."	Check your inbox (or the development console, during testing) for the verification email and follow its link, then try again.
Submitting a claim with missing evidence description	"Evidence description is required."	Provide a description of the identifying details that prove you own or found the item.
Submitting evidence photos over 5 MB each	"Each evidence photo must be no more than 5 MB."	Reduce the file size or choose a different photo.
Attempting to submit a claim after the match has been rejected	"This match is no longer available for claiming."	If you believe there is still a match, review the listings manually and contact the posting user if appropriate

		through other channels (this is outside the Lost and Found application).

17. Frequently Asked Questions

- **Why can't I create a listing right after registering?**
  Your email must be verified first. Check your inbox for the verification email and follow its link.
- **Can I see other people's private details on a Found item?**
  No. Private identifying details are visible only to the person who posted the listing. They are never included in public listings, search results, notifications, or anyone else's view unless they submit a claim for review.
- **Can someone else edit or delete my listing?**
  No. Only the account that created a listing can edit or delete it, regardless of what any screen shows.
- **What happens if I lose my password and never requested a reset?**
  Use Forgot your password? on the Login page at any time; it works even if you never used it before.
- **How many photos can I attach, and in what format?**
  Up to 3 photos per listing, each a JPEG, PNG, or WebP file no larger than 5 MB. For claim evidence, you can submit up to 2 evidence photos using the same formats and size limit.
- **Why did I receive a match notification for two items that don't look the same?**
  The matching engine compares category, location, date, and keyword similarity. Items in the same category at the same location posted near the same date are flagged as potential matches so you can review them. The score (0 to 1.0) reflects how confident the engine is. Always review the full details yourself before approving or rejecting a match.
- **What if I approve a claim but then realize it's the wrong person?**
  Once a claim is approved, the message thread opens and the listing is marked as claimed. If you realize there is a mistake, communicate with the claimant through the message thread. You can then decide whether to work out the situation, or if you need to escalate, contact campus security or the lost and found office directly.
- **Can I send messages before a claim is approved?**
  No. Message threads only open after a claim is approved. Before approval, you can review the claim evidence and accept or reject it, but you cannot send messages. Once approved, the thread opens and you can communicate.
- **Are messages saved if I delete my account or the listing?**
  During the course project, deletion removes the listing from public view, but records are kept internally. Messages in an approved claim thread are part of that record. If you have completed the handoff, you can delete the listing (which marks it as resolved) and the message thread will close.
- **Can I resubmit a claim if it is rejected?**
  No. Once you submit a claim and it is rejected, you cannot resubmit evidence for the same listing. You can, however, browse and search for other similar listings and submit claims to those instead.

18. Quick Reference: Typical Phase 2 Workflow

i. Register with a campus email address.
ii. Verify your email from the link sent to you.
iii. Sign in.

iv. Browse or search for an item.
v. If it is not already listed, create a Lost or Found listing and save it.
vi. Watch for match notifications. When a potential match is found, you receive a notification in the browser and via email.
vii. Review matches at your own pace from the Notifications page or from badges on your listings.
viii. If you believe a match is your item, submit claim evidence describing identifying details.
ix. If a claim is submitted to one of your listings, review the claimant's evidence and compare it to your private identifying details.
x. Approve the claim if the evidence matches. An in-app message thread opens.
xi. Use the message thread to coordinate the handoff (time, location, etc.).
xii. Once you have completed the handoff in person, delete or mark the listing as Resolved.
xiii. Sign out when finished.

5. Compilation and Run Instructions (Phase 2)

This document explains how a reviewer with repository access can download the Phase 2 source code from GitHub, compile and run the backend and frontend on a local machine, run the automated test suite, and open the HTML demo file that shows the Phase 2 workflow without any installation. Each step was written against the source files on the deliverable-4 branch.

1. Overview

The application is a MERN stack project with two independent Node.js projects in the repository: src/backend (Express API and MongoDB access) and src/frontend (React, built with Vite). Each has its own package.json and lockfile, so each is installed and started separately.
Phase 2 adds the matching engine (scores Lost and Found listing pairs and creates potential matches), the notification service (in-app notifications and optional email), the claim workflow (evidence submission and owner review), and in-app messaging for approved claims.
There are two ways to see the application. Sections 4 to 7 run the real backend and frontend. Section 9 describes the HTML demo file, which runs in any modern browser with no server, database, or installation step.

2. Prerequisites

- Git, to clone the repository (or a browser, to download a ZIP archive).
- Node.js 22 LTS. The code was built and tested with Node 22. Check the installed version with node --version.
- pnpm, the package manager the repository's lockfiles were generated with. Install it with npm install -g pnpm, then check with pnpm --version. npm also works for installation and was used for the team's build checks.
- MongoDB 7.0 or later, either a local instance (Docker is shown in Section 5) or a free MongoDB Atlas cluster. The backend tests need a reachable MongoDB server before they start.
- Optional: an SMTP account (Mailtrap, Ethereal, or similar). If SMTP is not configured, verification and notification emails are printed to the backend terminal instead of being sent.

3. Getting the Source Code

3.1 Option A: clone with Git (recommended)

The Phase 2 code is on the deliverable-4 branch, and an identical copy is in the src folder of the main branch.

```
git clone -b deliverable-4 https://github.com/wmirza1987/csce5430-falcons-
lostandfoundapp.git
cd csce5430-falcons-lostandfoundapp
```

3.2 Option B: download a ZIP archive

i. Open https://github.com/wmirza1987/csce5430-falcons-lostandfoundapp, select the branch menu, and choose deliverable-4.
ii. Select Code, then Download ZIP.
iii. Extract the archive. The extracted folder contains src/backend and src/frontend.

3.3 Where things are

Item	Location in the repository
Backend source (Express API, services, tests)	src/backend

Frontend source (React and Vite)	src/frontend
HTML demo file	The root of the deliverable-4 branch: D4 Item12 Lost and Found for Campus - Phase 2 Demo.html
Deliverable reports	reports/deliverable-4 (on the main branch)

4. Configuration

The deliverable-4 branch includes .env.example templates for the backend (src/backend/.env.example) and the frontend (src/frontend/.env.example). Copy each template to a file named .env in the same folder and edit the values as shown below. Never commit a .env file. The repository's .gitignore already excludes it.

4.1 Backend: src/backend/.env

```
PORT=5000
MONGODB_URI=mongodb://127.0.0.1:27017/lost_and_found
MONGODB_URI_TEST=mongodb://127.0.0.1:27017/lost_and_found_test
JWT_SECRET=replace-with-a-long-random-string
APP_BASE_URL=http://localhost:5173
CLIENT_ORIGIN=http://localhost:5173
MATCH_THRESHOLD=0.6
EVIDENCE_ENCRYPTION_KEY=
SMTP_HOST=
SMTP_PORT=
SMTP_USER=
SMTP_PASS=
SMTP_FROM=no-reply@lostandfound.unt.edu
```

Generate a random JWT_SECRET, and optionally a 32-byte EVIDENCE_ENCRYPTION_KEY (64 hex characters), with this command:

```
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
```

If EVIDENCE_ENCRYPTION_KEY is blank, the backend derives an evidence key from JWT_SECRET so the app still runs. Set the key explicitly for any deployed instance. Leave the SMTP_* values blank to log emails to the backend terminal.

4.2 Frontend: src/frontend/.env

```
VITE_API_BASE_URL=http://localhost:5000
```

Change this value only if the backend runs on a different port or address.

5. Start MongoDB

With Docker installed, run a local MongoDB 7.0 server bound to your own machine:

```
docker run -d --name lostfound-mongo --restart unless-stopped \
  -p 127.0.0.1:27017:27017 \
  -v lostfound-mongo-data:/data/db \
  mongo:7.0
```

Alternatively, set MONGODB_URI and MONGODB_URI_TEST to your MongoDB Atlas connection strings. In Atlas, add your current IP address under Network Access.

6. Backend: Install and Run

6.1 Install dependencies

```
cd src/backend
```

```
pnpm install
```

This installs Express, Mongoose, jsonwebtoken, bcrypt, multer, nodemailer, and, for development, Jest, Supertest, and nodemon, as listed in src/backend/package.json.

6.2 Start the API

```
pnpm dev        # development server with automatic restart
pnpm start      # plain Node.js run
```

The API starts on the port set by PORT (5000 by default). The terminal prints Server running on port 5000 once the server is listening.

6.3 Confirm it is running

Open http://localhost:5000/api/health in a browser. The response is JSON with status set to ok and the message Team Falcons backend app is running.

7. Frontend: Install and Run

7.1 Install dependencies and start the dev server

In a second terminal, with the backend still running:

```
cd src/frontend
pnpm install
pnpm dev
```

Vite prints a local address, normally http://localhost:5173. Open it in a browser. If the backend is not running, pages that call the API show an error and do not load data.

7.2 Production build and lint

```
pnpm build          # writes static files to src/frontend/dist
pnpm lint           # oxlint static checks
```

The team's last run of pnpm lint reported 0 errors and 29 warnings. The frontend has no automated test suite; the lint and build steps are the automated checks for the frontend.

8. Running the Automated Test Suite

8.1 Backend tests

The backend uses Jest and Supertest. The test setup connects to MongoDB at MONGODB_URI_TEST, or, if that is not set, to a database named lost_and_found_test on the same server as MONGODB_URI. It drops every collection before each test and drops the test database after the run. Do not point MONGODB_URI_TEST at a database you want to keep.

```
cd src/backend
pnpm test          # or: npm test
```

The expected result is 6 test suites and 83 tests, all passing:

Test file	Covers
claims.test.js	Claim submission, owner review, approval and reversal, evidence access
messaging.test.js	Thread access, message posting, contact-detail warning, paging, moderation flags
matchingService.test.js	Match scoring, threshold, background refresh, stale match removal
notificationService.test.js	In-app and email notifications, contact-detail removal

matchApi.test.js	Match listing endpoints and access rules
listingFixes.test.js	Listing update and delete behaviour that interacts with matching

If the tests hang, MongoDB is not reachable. Confirm that the backend connects with pnpm dev before running the tests.

9. HTML Demo File

The demo file is a single, self-contained HTML page. It is the quickest way to see the Phase 2 workflow, and it needs no server, database, or installation.

9.1 Where to find it

- On the deliverable-4 branch, at the root of the repository: D4 Item12 Lost and Found for Campus - Phase 2 Demo.html
- The same file is also in reports/deliverable-4 on the main branch, as D4 Item 12 Phase 2 Demo.html: https://github.com/wmirza1987/csce5430-falcons-lostandfoundapp/tree/main/reports/deliverable-4

9.2 How to open it

i. Download the file, or use the copy in the cloned repository.
ii. Double-click the file, or drag it into a browser window (Chrome, Edge, or Firefox).
iii. Sign in with one of the sample accounts: alex.demo@my.unt.edu or jordan.demo@my.unt.edu. The password for both is password.

9.3 What it demonstrates

The demo is a working front-end application, not a slide presentation. It covers sign-in, browsing listings, reporting a Found or Lost item, potential matches, submitting a claim, reviewing a claim, the message thread that opens after approval, and notifications.

9.4 How it differs from the running application

The demo does not connect to the backend or to MongoDB. Its data is simulated and kept in the browser. It shows the intended user flow and screens, so use Sections 6 to 8 to check that the real backend and frontend compile, run, and pass their tests.

10. End-to-End Walkthrough (HTML Demo)

This walkthrough describes the Phase 2 workflow from registration to an approved claim. The HTML demo file (Section 9) runs these steps in the browser. The steps have not yet been run against the live backend and frontend; that live run is planned for the final phase of the project. The same steps apply when you run the live system (Sections 6 and 7). It needs two accounts, so use a second browser profile or a private window for the second user.

i. Open the frontend and select Register. Create an account with a @my.unt.edu or @unt.edu address.
ii. Find the verification link printed in the backend terminal (or in your inbox, if SMTP is configured) and open it.
iii. Select Login and sign in.
iv. Select Create Listing, choose Found, fill in the required fields (title, category, description, date, building or zone), attach a JPEG, PNG, or WebP image, and submit.
v. Create a second account, sign in, and create a Lost listing with the same category, a nearby date, and a matching building.
vi. Open Notifications for the Lost listing's owner. If the matching engine found a pair, follow the potential match notification to the match page.

vii. On the match page, open the claim form, describe the identifying details in the evidence, and select Review claim. Then confirm the submission.
viii. Sign in as the Found listing's owner. Open the claim notification, examine the evidence, and submit the decision (approve or reject).
ix. After approval, open Messages, open the claim's thread, and send a test message. Sign in as the other user and confirm the message is shown.
x. In the current build, approving a claim also resolves the listing. There is no separate Mark Resolved step in the app.

11. Troubleshooting

Symptom	Likely cause	What to do
MongoDB connection failed on startup	MONGODB_URI is missing or malformed, the server is not running, or Atlas blocks the connection.	Confirm the server is running (Section 5). In Atlas, add your IP address under Network Access.
Port 5000 or 5173 is already in use	An earlier server process is still running.	Stop the other process, or change PORT in the backend .env and VITE_API_BASE_URL in the frontend .env to match.
Frontend pages show a connection error	The backend is not running, or VITE_API_BASE_URL points to the wrong address.	Start the backend, confirm http://localhost:5000/api/health responds, then restart the frontend dev server.
pnpm install fails	Wrong branch, a missing lockfile, or an old Node.js version.	Confirm you are on deliverable-4 and that pnpm-lock.yaml is present. Use Node.js 22 LTS. Then run pnpm install again.
Verification or notification emails never arrive	SMTP_* values are blank.	This is expected locally. Read the email links in the backend terminal.
pnpm test hangs or times out	MongoDB is not reachable from MONGODB_URI_TEST.	Start MongoDB, confirm pnpm dev connects, then run the tests again.
Startup error about JWT_SECRET or the evidence key	JWT_SECRET is missing, or EVIDENCE_ENCRYPTION_KEY is not 32 bytes.	Set JWT_SECRET. If you set EVIDENCE_ENCRYPTION_KEY, use 64 hex characters (Section 4.1).
No matches appear	The listings do not meet the match rules, or the score is below the threshold.	Use the same category, a date within 7 days, the same building or zone, and at least one shared keyword. For testing, lower MATCH_THRESHOLD (for example 0.4) and restart the backend.

12. Verification Status

The following was checked by the team and is recorded here so the reviewer knows what is and is not confirmed.

- Backend: npm test (6 suites, 83 tests) passed against a local MongoDB 8 server in a Linux environment, as reported by the team. These tests were not rerun for this review, because MongoDB is not installed on the review machine.
- Frontend: the production build completes (vite build, confirmed during this review), and the team's lint run reports 0 errors and 29 warnings.

- A recorded run of the Phase 2 workflow is available in the HTML demo file (Section 10). It runs in the browser on simulated data, so it does not exercise the live backend. Not yet recorded: a live run of the walkthrough against the running backend, and a run on Windows or on a second machine.

6. Code Inspection Feedback (Phase 2)

1. Source of This Feedback

Team Falcons participated in a reciprocal Code Inspection Workshop with Team Medley as part of the CSCE 5430 course structure. Both teams examined each other's Phase One implementation, with Team Medley focusing on security considerations, edge case handling, and design patterns in the Lost and Found application. Team Medley posed eight targeted questions about the implementation, with responses provided by Team Falcons members during the session. This feedback document addresses each concern raised, documents the team's disposition toward each item, and outlines planned actions for Deliverable 4 and beyond.

2. Code Inspection Feedback, Workshop Disposition, and Deliverable 4 Action

Concern / Suggestion	Raised By	Code Inspection Disposition	Action for Deliverable 4
NoSQL injection vulnerability in search input handling (Part 3, line 99 searching user input directly into database query)	Team Medley	Acknowledged as a genuine security vulnerability present in Phase One MVP. Team recognized this as beyond MVP scope but identified as critical for Phase Two.	Implement input validation and parameterized queries for all database searches in Phase Two. Add test cases (SEC-01 to SEC-09 in test plan) to verify injection prevention.
Account is saved to database before email verification is confirmed (Part 2, lines 41-43)	Team Medley	Acknowledged as a valid design question. Team Falcons noted the account record is created immediately and marked as unverified, pending email confirmation.	Evaluate whether to require verified email before account persistence or allow unverified accounts in the database during Phase Two. Document the chosen approach in Phase 2 design rationale.
Edge case handling for email delivery failures and bounced emails (when email provider fails or rejects a message)	Team Medley	Acknowledged that no edge case handling exists for email failures in Phase One. Team noted this is beyond MVP scope and would be addressed in Phase Two.	Implement email failure isolation (REL-03 to REL-04 in test plan) so that email delivery failure does not prevent core account or claim operations. Define fallback behavior for bounced notifications.
Inconsistent token variable naming (rawToken reused for multiple purposes instead of rawEmailVerificationToken, rawPasswordResetToken)	Team Medley	Team Falcons agreed this is a valuable code clarity suggestion that would improve maintainability. Noted this is a refactoring opportunity for Phase Two.	Rename token variables to be explicit (rawEmailVerificationToken, rawPasswordResetToken) during Phase Two backend refactoring. Update all references in auth service and tests.

Email addresses should be converted to lowercase before comparison or database query (Part 1, line 25 defines email lowercase: true, but forgotPassword does not check this condition)	Team Medley	Team Falcons acknowledged this is a correct observation. The lowercase rule is applied at schema level but not consistently enforced at query time.	Add email normalization (toLowerCase) before every email comparison and database lookup (authentication, password reset, registration). Add test case (UT-AUTH-12) to verify normalization.
Image file type validation should account for extension variants (JPG vs JPEG, PNG vs PNG, etc.)	Team Medley	Team Falcons confirmed the current implementation checks MIME type but acknowledged that file extension checking should handle common variants.	Enhance file upload validation in Phase Two to accept common file extension variants (jpg/JPG, jpeg/JPEG). Reference uploadMiddleware.js validation and FR-LIST-02 requirements.
Soft delete policy undefined: records are kept indefinitely but no retention or purging policy is documented (Part 3, line 224)	Team Medley	Team Falcons acknowledged soft delete serves both moderation (preserving records for review) and analytics (tracking common item categories). Noted no automatic purging policy exists.	Define and document a soft delete retention policy during Phase Two design. Consider GDPR/privacy implications and free tier storage constraints. Add policy documentation to the Compilation and Run Instructions.
Account save and verification flow timing: if email verification fails, is the account locked or left in an indeterminate state?	Team Medley	Team Falcons noted that unverified accounts can sign in but cannot create listings (FR-AUTH-01 enforcement in createListing). This is the current intended behavior.	Document this explicitly in Phase Two requirements. Add test cases (UT-AUTH-04) to verify unverified users cannot access protected operations. Clarify in the user manual that email verification is required before listing creation.

3. Items Requiring Deferred or Alternative Approach

No suggestions from the Code Inspection Workshop were rejected outright. All eight concerns raised by Team Medley were acknowledged as either valid Phase Two improvements or clarifications that improve code clarity. Some items are deferred as beyond Phase One MVP scope (email failure handling, NoSQL injection prevention, soft delete retention policy) but have been explicitly mapped to Phase Two deliverables and test cases. The team's approach is to treat the Phase One MVP as a foundation and address security hardening, edge case handling, and data management policies in Deliverable 4 (Phase Two implementation) and Deliverable 5 (Phase Three).

4. Summary

The Code Inspection Workshop identified seven concrete improvement areas and one design clarification for Team Falcons' Lost and Found implementation. Of these eight items: zero were rejected, all eight were acknowledged as valid, four are mapped directly to Phase Two requirements and test cases (NoSQL injection prevention, email failure handling, email normalization, file extension variants), two are deferred as design decisions (account persistence timing, soft delete retention policy), and two are code clarity improvements identified for Phase Two refactoring (token variable naming, verification flow documentation). Team Falcons acknowledged that several security hardening measures and edge case handling scenarios, particularly around email delivery and database input validation, should be prioritized in Phase Two development. The workshop feedback has been incorporated into the Phase Two requirements document and test case plan, ensuring that all identified concerns are formally tracked and scheduled for implementation.

7. Reflection (Phase 2)

What Has Been Accomplished

Deliverable 4 is the team's second implementation phase, extending the Phase 1 core listing workflow with four major feature subsystems: the matching engine, claim verification workflow, notification service, and in-app messaging. All fifteen required items were completed: a detailed Phase 2 requirements document expanding every functional, nonfunctional, and interface requirement for the four new subsystems carried forward from Deliverable 2; a full UML design package with class, sequence, and use case diagrams modeling the Match, Claim, Message, and Notification entities alongside Phase 1's User and Listing classes, including normal paths and two error cases each; a 104 case test plan covering matching engine scoring, claim evidence submission and verification, notification delivery across in-app and email channels, messaging thread management, privacy boundaries, security validation, and integration across all four subsystems; working Phase 2 source code for the backend matching, claim verification, notification, and messaging services and the corresponding React frontend interfaces; an updated user manual documenting the matching notification workflow, claim submission and review forms, messaging interface, and new on-screen messages; updated compilation and run instructions with Phase 2 environment setup, test execution, and end-to-end workflow steps; a coding and AI usage disclosure documenting tools, prompts, and verification procedures for Phase 2 code generation; a code inspection feedback document addressing all eight concerns raised by Team Medley, with explicit mappings to Phase 2 improvements and Phase 3 enhancements; updated meeting minutes capturing architecture decisions and integration approach; the member contribution table; and the supporting repository artifacts including the note-deliverable-4 file and the core functionalities match report. The team extended a working Phase 1 system through four new major features to reach the end of Phase 2 with a complete claim to item recovery workflow end to end.

What Went Well

- The verification practices established in Deliverable 3 were applied consistently to Phase 2. Matching engine scoring logic generated by AI tools was tested against sample datasets before integration. Claim evidence validation logic was checked against the FR-CLM-01 to FR-CLM-03 requirements. Notification service queuing and delivery patterns were verified to handle both in-app and email delivery paths. This verification approach caught real issues: an initial version of the matching score calculation had category matching weighted too heavily, the claim evidence storage schema initially lacked nullable fields for optional data, and notification cleanup processes needed explicit dequeue sequencing to prevent race conditions.
- Requirement traceability was maintained across all fifteen items. Every Phase 2 requirement from Deliverable 2 is referenced explicitly in the Phase 2 requirements document, the UML diagrams name the corresponding entities and services, the test cases link to requirement IDs in their coverage matrix, and the user manual describes the workflows defined in the requirements. A reviewer can follow a single requirement through matching engine scoring (FR-MATCH-02), claim evidence (FR-CLM-01), notifications (FR-NOTIF-01), or messaging (FR-COM-01) across every artifact.
- The team responded to Code Inspection Workshop feedback by acknowledging all eight concerns raised by Team Medley and recording a planned action for each one in the disposition table in Section 2. NoSQL injection prevention is planned to be implemented for the search endpoints and has not yet been implemented. Email normalization (toLowerCase) and email delivery failure isolation are also planned, not yet implemented. The matching engine and notification service were designed with explicit integration points defined in the sequence diagrams before coding began, reducing the rework needed during the D3 integration phase.

- The core versus stretch feature split from Deliverable 1 was applied to Phase 2 scope. Core Phase 2 features (matching engine with category and location matching, simple claim evidence collection, in-app notifications, basic in-app messaging) are complete. Stretch features (weighted claim scoring variants, image-based claim evidence, sentiment analysis on messages, delivery receipts for notifications) are identified for Phase 3 but do not block Phase 2 completion.
- Schedule pressure was managed better than in Deliverable 3. An internal target of a complete Phase 2 submission by November 6, three days ahead of the November 9 due date, was set at the start of the deliverable and held throughout. This slack absorbed the rework that verification revealed without compressing the final submission timeline.

What Could Be Improved

- Phase 2 scope, with four major subsystems, created integration complexity that exceeded Phase 1's scale. The matching engine, claim workflow, notification service, and messaging system all interact, and testing the interactions required more end-to-end integration test cases than the 104 unit and integration tests initially planned. Adding more explicit testing of cross subsystem failures (e.g., matching engine surfaces a match but notification delivery fails) earlier in the phase would have reduced defects found during final code review.
- The notification service added a new architectural pattern: asynchronous event driven delivery with queues. This pattern was not fully tested for race conditions (multiple notifications queued for the same listing, notification cleanup running while new notifications are being added). Building a dedicated notification integration test harness earlier, rather than relying on end-to-end system testing, would have surfaced these concurrency issues sooner.
- Some enhancements acknowledged during the Code Inspection Workshop were deferred to Phase 3 but would have been valuable in Phase 2. Weighted claim scoring (combining evidence image count, detail specificity, and claimant history) was deferred, as was structured claim detail fields (separating different types of identifying evidence). These would strengthen the claim verification workflow but required additional UX design and backend schema changes beyond the Phase 2 scope. Documenting why these were deferred and where they fit in Phase 3 more explicitly in the deliverable would make the deferral decisions more visible.
- Dependency version management continued to be error prone. Package.json files for both frontend and backend required manual audit to ensure compatibility across Node.js version, React versions, Express, and database driver versions. Adding a automated dependency check or a pinned lock file verification step as part of the code review process (in addition to the manual audit already done) would catch version conflicts earlier.
- The messaging interface in the React frontend required more iteration than anticipated to handle the asynchronous polling pattern (clients refresh every 15 seconds rather than using a real time socket). Initial implementations had unnecessary re renders and unnecessary API calls on every refresh cycle. Optimizing React state management and memoization to reduce unnecessary refreshes should be a standing practice in Phase 3, not an end-of-phase optimization.

Looking Ahead

Deliverable 5 builds on the Phase 2 foundation with three remaining major features: the campus map view (plotting listings by building or zone), the admin and analytics dashboard (for campus security and lost-and-found office staff), and full role based access control (Moderator and Administrator roles). The matching engine, claim verification, and in-app messaging systems implemented in Phase 2 are the backbone of the claim-to-recovery workflow; Deliverable 5 focuses on visibility and operational management through the map and admin interfaces. The verification and traceability practices

established in Deliverable 3 and carried through Deliverable 4 will remain in place for Phase 3: checking AI-generated code against requirements before committing, maintaining requirement IDs across all artifacts, and building schedule buffer into the timeline. The identified improvements from the Code Inspection Workshop, particularly NoSQL injection prevention, email failure isolation, and email normalization, remain in place as Phase 3 foundations. Beyond the immediate deliverable, the deferred enhancements (weighted claim scoring, structured claim detail fields, notification delivery receipts) are candidates for post-delivery refinement if the team continues beyond the semester's formal end.

8. Member Contribution Table

Contribution weighting for this deliverable reflects each member's role and scope of work across the fifteen required items, detailed below.

Member Name	Contribution Description	Overall Contribution (%)	Note
Waleed Mirza	Coordinated the overall Deliverable 4 effort across all fifteen required items. Authored and detailed the Requirements (Phase 2) document, expanding every functional, nonfunctional, and interface requirement for the four Phase 2 subsystems (matching engine, claim verification, notifications, messaging) carried forward from Deliverable 2. Reviewed and corrected the UML Design Package (Phase 2), rebuilding the sequence diagram to model the complete claim verification workflow from match notification through claim evidence submission, owner review, and approval or rejection, and ensuring all new entities (Match, Claim, Message, Notification) aligned with Phase 2 requirements. Reviewed the Test Cases document (Phase 2), closed coverage gaps in cross subsystem integration testing, and verified the 104 test case catalog. Authored the Code Inspection Feedback document, mapping all eight concerns Team Medley raised to concrete Phase 2 implementation and Phase 3 refinements. Authored the Reflection document, synthesizing the Phase 2 experience and looking ahead to Phase 3. Contributed to the backend implementation, including the claim verification workflow service and the integration coordination between matching engine, claim service, and messaging service. Compiled, formatted, and assembled every Deliverable 4 document. As Project Management Lead, coordinated the team schedule and deliverable submission deadlines.	35%	Not applicable
Venkata Sai Anirudh Naidu Mamidi	Implemented the Phase 2 backend services: the matching engine service with category, location, and keyword similarity scoring; the claim verification service handling evidence submission, storage, and status transitions; the notification service with asynchronous queue management, in-app and email delivery patterns, and delivery status tracking; and	25%	Not applicable

	integration of all three services with the Phase 1 listing and authentication controllers. Updated the MongoDB schema with Match, Claim, Message, and Notification models. Managed the GitHub repository configuration and branch structure for Phase 2 integration as Configuration Management Lead. Updated the Compilation and Run Instructions with Phase 2 environment setup and end-to-end workflow verification steps.		
Sai Sathvika Potluri	Produced the first draft of the UML Design Package (Phase 2), including the class diagram modeling the four new Phase 2 entities and services, the sequence diagram showing the claim verification workflow from initial match through evidence submission and owner review, and the use case diagram capturing Claimant and Finder roles interacting with the match, claim, and messaging subsystems. Produced the first draft of the User Manual (Phase 2), documenting the matching notification interface, claim submission and evidence upload forms, claim review side by side comparison, and in-app messaging interface, including 13 figure placeholders for screen illustrations. Contributed to design review and documentation consistency across all Deliverable 4 items as Design and Documentation Lead.	20%	Not applicable
Sai Asritha Thati	Authored the Test Cases document (Phase 2), covering the planned test catalog across functional, boundary, security, privacy, accessibility, and integration categories. Implemented 104 total test cases (UT-MATCH-01 to 14, UT-CLM-01 to 18, UT-NOTIF-01 to 12, UT-MSG-01 to 16, UT-AUTH-01 to 08, UT-PRIV-01 to 12, INT-01 to 24) with specific requirement traceability, preconditions, test actions, and expected outputs. Implemented the Phase 2 React frontend, including the notifications page showing match alerts and claim events, the match review interface for comparing listing details, the claim submission form with evidence upload, the claim review interface for owners, and the in-app messaging interface with message history and send functionality. As Front End Implementation	20%	Not applicable

	Lead and Testing Lead, led unit and integration testing of all frontend components and coordinated testing with the backend services.		
Total		100%	

9. Coding and AI Usage Disclosure

0. How to Read This Section

AI assistance was used in preparing this deliverable, and will be used in developing the Phase 2 code. This section identifies AI generated versus self written code, the tools used, representative tasks and prompts, the team's validation process, and specific mistakes the team found and corrected. This section is organized in two parts.

Section A is the disclosure required by the Deliverable 4 rubric: AI usage specific to the Phase 2 source code. Phase 2 implements the matching engine, claim verification workflow, in app messaging, and notification service. The tools and process for each module are fixed below and will not be marked N/A, since the team has already committed to using AI for coding; only the specific defects found and fixed will be added as each module is completed.

Section B goes beyond what the rubric requires. Because the team also used AI extensively while preparing the written parts of this deliverable (requirements, UML diagrams, test cases, user manual, and meeting minutes), Section B discloses that usage too, in the same level of detail, for full transparency.

1. AI Tools in Use

Waleed uses Claude (Anthropic, Sonnet model family, accessed through claude.ai) for the Requirements Phase 2, UML Design Package review and correction, Test Cases review, User Manual review, Meeting Minutes, and this disclosure itself. Anirudh, Sathvika, and Asritha use Claude and ChatGPT for their assigned coding and documentation tasks: the Phase 2 backend implementation (matching engine, claim service, messaging service, notification service), Phase 2 frontend implementation (claim forms, messaging interface, owner review interface), and documentation updates.

2. Section A: Code AI Usage Disclosure (Required)

2.1 Code Authorship Summary

Module	AI Tool Used	Authorship (AI Generated / Self Written)	Status
Matching engine (FR MATCH 01 to 03)	Claude and ChatGPT	AI generated, human reviewed and tested	In Progress
Claim verification (FR CLM 01 to 03)	Claude and ChatGPT	AI generated, human reviewed and tested	In Progress
In app messaging (FR COM 01 to 03)	Claude and ChatGPT	AI generated, human reviewed and tested	In Progress
Notification service (FR NOTIF 01 to 02)	Claude and ChatGPT	AI generated, human reviewed and tested	In Progress
React frontend Phase 2 components	Claude and ChatGPT	AI generated, human reviewed and tested	In Progress
MongoDB schema extensions	Claude and ChatGPT	AI generated, human reviewed and tested	In Progress

2.2 Detailed Code AI Usage Log

The rows below record each coder's committed tool and process, agreed at the Deliverable 4 planning meeting. The Mistakes/Hallucinations column will be filled in with specific defects as code is written and tested; the process by which mistakes are found and handled is fixed and stated below.

Team Member	Tool	Task and Example Prompt	AI Generated vs Self Written	Mistakes/Hallucinations Found and Fixed
Anirudh	Claude and ChatGPT	Generate the Phase 2 backend modules: MatchingService with scoring logic, ClaimService for evidence submission and review, MessagingService for in app message threads, NotificationService for queueing and delivery. Example prompt: "Write a scoring function that combines category match, location proximity, date range overlap, and keyword overlap into a single match confidence score between 0 and 1."	AI generated, human reviewed and tested	To be filled as coding completes and defects are found.
Asritha	Claude and ChatGPT	Generate the Phase 2 React frontend components: ClaimSubmissionForm, OwnerClaimReviewInterface, MessagingThread, NotificationCenter. Example prompt: "Create a React component that displays a Found listing alongside its private identifying details, and renders a form for the claimant to submit matching evidence with text input and optional image upload."	AI generated, human reviewed and tested	To be filled as coding completes and defects are found.
Waleed (coordinating)	Claude and ChatGPT	Schema extension and integration review. Example prompt: "Audit the Match, Claim, Message, and Notification collection schemas against the Phase 2 requirements and the UML class diagram; identify any missing fields or constraints."	AI generated, human reviewed and tested	To be filled as integration testing completes.

3. Section B: AI Usage in Report Preparation (Supplementary)

Not required by the rubric, included for full transparency since AI was used extensively in preparing the written sections of this deliverable.

Team Member	Tool	Task and Example Prompt	AI Generated vs Self Written	Validation Performed
Waleed	Claude (Anthropic, Sonnet, via claude.ai)	Expand the Phase 2 functional and nonfunctional requirements from Deliverable 2, Section 9.4 and Section 6, adding implementation detail and verification guidance for each requirement. Example prompt: "Add implementation detail and verification approach for FR MATCH 01, explaining how the matching engine scores and ranks matches by category, location, date proximity, and keyword overlap."	AI generated first draft from the Deliverable 2 requirements text	Every requirement ID was cross checked against the Deliverable 2 tables; all Phase 2 scope (matching, claim, messaging, notification) was verified to be present; the structure matched the Phase 1 requirements document for consistency.
Waleed	Claude	Review the UML Design Package (class, sequence, use case diagrams) for Phase 2, checking against the Phase 2 requirements. Example prompt: "Check that the class diagram includes the Match, Claim, Message, and Notification classes, that relationships are labeled, and that private fields are marked with minus visibility."	Class and sequence diagrams were Sathvika's first draft; use case diagram regenerated after review	Every diagram was checked element by element against the UML Design Phase 2 document and the Phase 2 functional requirements; relationship arrows and labels were verified to match the requirement traces.

Waleed	Claude	Review the test case document against the Phase 2 requirements. Example prompt: "Audit the Phase 2 test cases for coverage of the matching engine scoring, claim evidence submission and review, message threading, and notification delivery. Identify any requirement areas with less than 3 test cases."	AI reviewed the initial test case catalog for completeness and gaps	All Phase 2 functional and nonfunctional requirements were traced to at least one corresponding test case ID; coverage gaps identified during this review are being addressed in the working test case draft.
Waleed	Claude	Using meeting transcripts and team decisions, generate updated meeting minutes. Example prompt: "Summarize the Deliverable 4 kick off meeting, including the matching engine scoring algorithm decisions, the claim evidence format, and the messaging thread structure."	AI generated the full discussion summary and action items based on meeting notes	The summary was reviewed against the source transcript; every action item was confirmed to match what the team decided; no invented decisions were added.
Waleed	Claude	Edit the Deliverable 4 planning spreadsheet to track the 15 required items against the Phase 2 scope and the team's work plan. Example prompt: "Update the point allocation and dependency graph to reflect the current status of Requirements	AI edited the existing spreadsheet directly using calculated fields and formula updates	The point total formula was verified to sum to 101 points; the dependency graph was checked to confirm all items are accounted for; no rows were added or removed.

		Phase 2 (complete), UML Design Phase 2 (complete), and the remaining items."		

4. Validation Standard

Consistent with the team's working process established at the Deliverable 3 planning meeting on September 30, 2026, and carried forward into Deliverable 4, no AI generated content, whether code, diagrams, or text, is accepted without being reviewed, and in the case of code, actually run and tested, by a team member. An AI first draft is treated as a starting point requiring engineering judgment, not a final deliverable.

For Phase 2 code, this means: each backend module (matching engine, claim service, messaging, notifications) is tested in isolation using unit tests before integration; each frontend component is checked against the UML Design Phase 2 and the user interface requirements before submission; and all four modules are tested end to end through the claim verification workflow before Deliverable 4 is marked complete.

10. Code Compiles and Runs

The Phase 2 backend (Express/Node.js) and frontend (React/Vite) source code has been uploaded to the project GitHub repository, in the src/backend and src/frontend folders of the deliverable-4 branch, and identical copies are in the src folder of the main branch: https://github.com/wmirza1987/csce5430-falcons-lostandfoundapp/tree/deliverable-4

As discussed with the course TA, an HTML based demo application, D4 Item 12 Lost and Found for Campus - Phase 2 Demo.html, is available as the Phase II working program. It is a single, self-contained HTML file that exercises the Phase 2 workflow in the browser, requiring no server, database, or install step. Its data is simulated in the browser; the live backend and frontend are covered by Section 5 (Compilation and Run Instructions). The demo file is available at: https://github.com/wmirza1987/csce5430-falcons-lostandfoundapp/tree/main/reports/deliverable-4 and at the root of the deliverable-4 branch: https://github.com/wmirza1987/csce5430-falcons-lostandfoundapp/tree/deliverable-4

11. Code Functionalities Match Report (Phase 2)

This report was produced by reading the Phase 2 source code directly against the Deliverable 4 Requirements document, and it is supported by the backend test suite results the team reported (83 tests in 6 suites). It has not been confirmed by running the full application in a browser; see the status note in Section 1 and the Compilation and Run Instructions document.

i. Basis and Limitation of This Report

This report compares what the Deliverable 4 Requirements document describes for Phase 2 against what the backend and frontend source code implements, requirement by requirement, with a specific file and function cited as evidence for each row. It covers the Phase 2 functional areas (matching, claim verification, notifications, and in-app messaging), the supporting nonfunctional requirements, and the supporting interface requirements that Phase 2 builds on, so that anything still open is stated explicitly.

This match rests on two kinds of evidence. The first is a static review of the committed source files. The second is a run of the backend test suite against a local MongoDB instance, as reported by the team, in which 83 tests across 6 suites all passed; that run was not repeated for this review; the frontend production build also completes without errors (vite build, confirmed during this review). Neither is a live run of the full application in a browser. The HTML demo file records a run of the Phase 2 workflow in the browser, but it uses simulated data and does not exercise the live backend. Code Compiles and Runs, and any row marked Not Verified, should be confirmed by following the Compilation and Run Instructions document before final submission.

Three points need a team decision before submission. The evidence limit is 2 photos in the requirements and the user manual but 3 in test cases UT-CLM-02, UT-CLM-12, and UT-CLM-13; the implementation uses 2. Approval resolves the listing as FR-CLM-02 states, whereas the user manual describes a separate Mark Resolved step; the implementation follows FR-CLM-02 and has no separate step. Match details are stored in the database rather than written to a log, because UT-PRIV-11 forbids logging them while FR-MATCH-03 asks for a log.

ii. Summary

Of the twenty-seven rows evaluated, twenty-five are implemented in full or in part: nineteen as described, and six partially, with the gap stated in each partial row. The other two cannot yet be verified without a live browser run or a load test (NFR PERF 01 and NFR COMP 01). The Phase 2 core is in place for matching, claim submission and review, opening the messaging thread on approval, notifications, and messaging. No row is implemented in a way that contradicts the Requirements document; where the implementation differs in naming or storage, the row says so.

Several features described in the user manual have no implementation and do not appear in the Requirements document, so they are not rows above: owner acceptance or rejection of a match, re-enabling matching for a listing, the admin claim report (UT-CLM-11 and UT-AUTH-06), notification batching (UT-NOTIF-12), performance tests (UT-MATCH-10 and UT-MATCH-11), and integration tests INT-16 and INT-17. Each should either be built or listed as out of scope in the consolidated report.

iii. Requirement by Requirement Match

Requirement	Described in Report As	Evidence in Code	Status
FR MATCH 01	Scores new and edited listings by category, campus location, date proximity, and keyword overlap, and notifies both parties of likely matches.	matchingService.js: WEIGHTS (category 0.35, date 0.25, location 0.20, keywords 0.20), calculateMatchScore, DEFAULT_MATCH_THRESHOLD of 0.6 (overridable with MATCH_THRESHOLD). Date credit is full within 1 day and half within 7 days. notifyPotentialMatch alerts both owners.	Implemented
FR MATCH 02	Recomputes matches when a listing is created or edited, and removes them when a listing is deleted.	listingController.createListing and updateListing call scheduleMatchRefresh; deleteListing calls removeMatchesForListing. Matching runs as a background job (pendingMatchJobs), so the listing request does not wait for it.	Implemented
FR MATCH 03	Records how each component contributed to a match score, for debugging, without exposing it to users.	Match.factors stores the category, date, location, and keyword contributions in MongoDB; matchController returns no factors. Deviation: the detail is stored in the database instead of a log, because UT-PRIV-11 forbids logging.	Partially Implemented

FR CLM 01	A claimant submits identifying evidence for a match: text of up to 1000 characters and up to two images.	claimController.submitClaim; POST /matches/:id/claims with handleEvidenceUpload (2 images, 5 MB each, JPEG, PNG, or WebP). evidenceCrypto.js encrypts the text and images with AES-256-GCM. Claim.explanationCiphertext and evidenceImages are select: false. ClaimForm.jsx includes a preview step.	Implemented
FR CLM 02	The original poster reviews the evidence beside the private listing details, then approves or rejects the claim.	claimController.decideClaim (PATCH /claims/:id/decision). Approval sets the listing to resolved and closes competing pending claims; rejection leaves the listing active. ClaimReview.jsx shows the evidence and the private details side by side. Deviation: the function is named decideClaim rather than reviewClaim.	Implemented
FR CLM 03	An approved claim opens an in-app messaging thread between the claimant and the original poster.	decideClaim calls createThread after the claim and listing are updated, and reverts both if the thread cannot be created. notifyThreadOpened alerts the claimant and the listing owner. Covered by UT-CLM-16 and INT-20 in claims.test.js.	Implemented
FR NOTIF 01	Sends in-app and email notifications for matches, claim submissions, claim decisions, and new messages.	notificationService.createNotification saves the in-app record before any email, and records the email status as pending, sent, failed, or skipped. Triggers: notifyPotentialMatch, notifyNewClaim, notifyClaimDecision, notifyNewMessage. A failed email is logged and never reverses the action that triggered it. Email delivery depends on SMTP settings and has not been tested against a live provider.	Implemented

FR NOTIF 02	Notifications contain no email addresses, phone numbers, or other contact details.	removeContactDetails is applied to notification text and email bodies. Templates use generic references and role labels only. Covered by notificationService.test.js (12 tests).	Implemented
FR COM 01	A one-to-one thread for each approved claim, with message history paginated at 50 per page.	messagingService.createThread, getThreadMessages (PAGE_SIZE 50, shown in timestamp order), and listThreads. Messaging.jsx and MessageThreads.jsx present the thread. Only the claim's participants can read it. Messages are text only.	Implemented
FR COM 02	Participants see role labels, not contact details or user IDs.	messagingService.roleFor and labelFor return Claimant, Finder, or Moderator; toMessageView omits sender identifiers. Messaging.jsx displays only these labels.	Implemented
FR COM 03	Detects phone numbers and email addresses in message text and warns the sender.	messagingService.containsContactDetails and CONTACT_WARNING. postMessage stores the message and returns the warning. Deviation: detections are not logged for later moderation review.	Partially Implemented
NFR PERF 01	95 percent of ordinary requests complete within 2 seconds with 50 concurrent users.	Matching runs in the background, so listing requests do not wait for it. No load test has been run, and performance tests UT-MATCH-10 and UT-MATCH-11 have not been built.	Not Verified
NFR SEC 01	HTTPS in production, and an authenticated session on every protected request.	Every Phase 2 route (matchRoutes, claimRoutes, messageRoutes, notificationRoutes) uses the protect middleware. HTTPS is a deployment setting and has not been checked.	Partially Implemented

NFR SEC 03	Validates request fields and upload limits, and rate limits sign-in, password reset, claim, and message requests.	authLimiter (authRoutes), claimLimiter (claim submission and decisions) and messageLimiter (message sending) through rateLimitMiddleware.js. Upload type and size are checked. Limits are held in memory per process, and no automated test checks for a 429 response.	Implemented
NFR PRIV 01	Public responses and logs exclude passwords, tokens, emails, private listing details, claim evidence, and message contents.	select: false on Claim evidence fields, Message flagReason, Notification emailBody, Listing privateDetails, and the User secret fields. Log statements record error messages only. The development notice for unconfigured email (emailService.js) prints no recipient or link.	Implemented
NFR REL 01	A write is confirmed before success is reported, and multi-step changes never leave partial state.	decideClaim awaits each save and reverts the listing and claim if createThread fails (UT-CLM-16). Matching failures are caught, so listing creation still succeeds.	Implemented
NFR USA 01	Required fields are identified, entered values are kept after an error, and each error appears beside its field.	ClaimForm.jsx and Messaging.jsx keep the typed text after a validation error and show messages near the field. Required-field marking is not yet consistent across the Phase 2 forms, and no usability test has been run.	Partially Implemented
NFR ACC 01	WCAG 2.1 AA for keyboard use, visible focus, labels, error identification, and colour use.	Phase 2 forms have labels and aria-describedby links, and App.css adds visible focus styles. No accessibility audit has been run, and the contrast of the status badges has not been checked.	Partially Implemented
NFR COMP 01	Works in current Chrome, Edge, Firefox, and Safari, from a 360 px viewport up to desktop size.	Responsive rules for the two-column claim review and the message bubbles were added in App.css, including a 360 px breakpoint. Not tested in any browser.	Not Verified

NFR MNT 01	README covers setup, environment variable names, test commands, and deployment, without secret values.	README documents EVIDENCE_ENCRYPTION_KEY and MATCH_THRESHOLD, the backend test command, and the frontend build and lint commands. private/ is git ignored.	Implemented
NFR TST 01	Automated tests cover authorization boundaries and the main success and failure paths for matching, claims, and messaging.	Six backend suites with 83 passing tests: claims (28), messaging (19), matchingService (13), notificationService (12), matchApi (6), and listingFixes (5). There are no frontend or end-to-end tests and no rate-limit test.	Partially Implemented
IF UI 03	Match alerts show the listing they relate to and link directly to the claim form.	notifyPotentialMatch sets actionPath to /matches/:id/claim for the lost owner. Notifications.jsx renders each actionPath as a link, and the email body contains the same link (buildEmailBody).	Implemented
IF UI 04	The claim form shows the matched listing, accepts text and images, and previews the claim before submission.	ClaimForm.jsx loads the match from GET /api/v1/matches/:id, shows the matched listing, collects text and up to two photos, and provides a review step before submitting.	Implemented
IF UI 05	The claim review page shows the listing's private details beside the claimant's evidence.	ClaimReview.jsx places the private details and the claimant's evidence side by side. Images are fetched with authorization from /claims/:id/evidence/:index.	Implemented
IF UI 06	The messaging interface shows a one-to-one thread, labels participants by role, and provides a composer.	Messaging.jsx shows role labels, a composer with a 1000-character limit, and the contact-detail warning.	Implemented
IF SW 04	The React client calls versioned Express endpoints for matches, claims, and messages, using JSON except for image uploads.	app.js mounts the routes under /api and /api/v1. matchRoutes, claimRoutes, and messageRoutes return JSON; only the evidence upload uses multipart form data.	Implemented

IF COM 04	Email notifications use the configured provider and link to the relevant in-app action.	notificationService.buildEmailBody appends a link built from APP_BASE_URL and the actionPath. emailService.js sends mail through the SMTP settings in the environment.	Implemented

12. Code in Correct Repository Folder

The Phase 2 backend and frontend source code is placed in the designated source code directories (src/backend and src/frontend) of the project repository on the deliverable-4 branch, with identical copies in the src folder of the main branch: https://github.com/wmirza1987/csce5430-falcons-lostandfoundapp/tree/deliverable-4

13. Core Inspection Report (Phase 2)

The Phase 2 Core Inspection Report (D4 Item 15 Code Inspection Report Phase 2) has been uploaded to GitHub and is accessible at: https://github.com/wmirza1987/csce5430-falcons-lostandfoundapp

14. Updated Meeting Minutes

Updated meeting minutes for Deliverable 4 have been uploaded to GitHub and are accessible at: https://github.com/wmirza1987/csce5430-falcons-lostandfoundapp/tree/main/meeting-minutes/deliverable-4

15. Note-deliverable-4 File

The note-deliverable-4 file, listing the name and repository location of every file submitted for this deliverable, has been uploaded to GitHub and is accessible at: https://github.com/wmirza1987/csce5430-falcons-lostandfoundapp